DevOn – SRE Interview Preparation

Senior SRE / DevOps / Reliability Engineering

JD-specific questions, spoken answers, production scenarios, technical pointers and rapid-revision cheat sheets

Candidate: Saroj Kumar Nayak

Interview focus: CI/CD • Release Management • SRE Metrics • Incident Management • Terraform • Observability • Scripting • Scaling • Resilience • Kubernetes • Cloud • ITIL • Application Reliability

How to use this guide

- Read the Spoken answer first. These are designed to be delivered naturally in roughly 30–60 seconds.
- Use the Specific pointers when the interviewer asks “how exactly?” or asks for implementation detail.
- For production scenarios, answer using: Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent.
- Keep examples grounded in real SRE/DevOps work. Do not claim tools, systems or outcomes you have not actually used.
- For this JD, repeatedly connect your answers to predictable releases, minimal production disruption, proactive monitoring, automation and reliability.

Core JD themes: cross-functional product teams, reliability-as-expertise, KPI reporting, production troubleshooting, preventive monitoring, housekeeping automation, on-call support, GitHub Actions, Azure/GCP, Terraform/Ansible, Splunk/Grafana, Python, React/React Native/Node.js, API-based microservices, ITIL/ServiceNow, performance engineering, managed Kubernetes (AKS/GKE).

1. Executive Interview Cheat Sheet

Your positioning: Present yourself as a production-focused SRE who combines reliability engineering, cloud/platform operations, CI/CD, Kubernetes, IaC, observability and automation. The DevOn JD is not asking only for a tool operator; it emphasizes predictable operations and continuous reliability improvement.

Area	What interviewer wants to hear	Keywords to use naturally
CI/CD & Release	Safe, repeatable releases with automated quality gates, rollback and traceability.	GitHub Actions, build/test/scan/package/deploy, approvals, canary/blue-green, rollback
SRE Metrics	Reliability measured from user/business perspective, not just infrastructure health.	SLI, SLO, SLA, error budget, burn rate, availability, latency, MTTR
Incidents	Fast mitigation first; systematic diagnosis; learning after recovery.	P1/P2, incident commander, timeline, logs/metrics/traces, RCA, corrective/preventive action
Terraform	Versioned, reviewed, repeatable infrastructure with controlled state.	modules, remote state, plan/apply, locking, variables, outputs, drift, CI validation
Observability	Correlate metrics, logs and traces to reduce MTTD/MTTR.	golden signals, dashboards, alert quality, correlation IDs, actionable alerts
Scaling	Scale based on measured bottlenecks and demand; protect dependencies.	HPA, cluster autoscaling, caching, queues, DB capacity, load testing
Resilience	Design for failure and controlled degradation.	timeouts, retries, backoff, circuit breaker, bulkhead, health probes, DR

Golden production answer pattern: “First I establish customer impact and scope. Then I check recent changes and the key service indicators, correlate metrics with logs and traces, isolate the failing component or dependency, mitigate to restore service, and only after stabilization complete RCA and preventive actions.”

2. CI/CD Release Concepts & Release Management

1. What is your approach to release management in an SRE environment?

Spoken answer: I treat release management as a reliability control, not just a deployment activity. I want every release to be traceable, tested, observable and reversible. I define entry criteria such as successful automated tests, security and quality checks, artifact immutability and environment readiness. For production I use controlled rollout, monitoring and a clear rollback path. After release, I validate service-level indicators and customer impact rather than only checking whether the deployment succeeded.

Specific pointers:

- Git commit → build → test → scan → package → artifact repository → deploy → smoke test → progressive rollout → monitor → release verification.
- Separate deployment from release where useful; feature flags can reduce blast radius.
- Rollback must be technically tested, not merely documented.
- Use change records/approvals where ITIL controls require them.

Likely follow-up: How do you decide whether a release is safe to continue?

2. CI/CD Release Concepts & Release Management

1. What is your approach to release management in an SRE environment?

Spoken answer: I treat release management as a reliability control, not just a deployment activity. I want every release to be traceable, tested, observable and reversible. I define entry criteria such as successful automated tests, security and quality checks, artifact immutability and environment readiness. For production I use controlled rollout, monitoring and a clear rollback path. After release, I validate service-level indicators and customer impact rather than only checking whether the deployment succeeded.

Specific pointers:

- Git commit → build → test → scan → package → artifact repository → deploy → smoke test → progressive rollout → monitor → release verification.
- Separate deployment from release where useful; feature flags can reduce blast radius.
- Rollback must be technically tested, not merely documented.
- Use change records/approvals where ITIL controls require them.

Likely follow-up: How do you decide whether a release is safe to continue?

2. What is the difference between deployment and release?

Spoken answer: Deployment is the technical act of putting a version into an environment. Release is making functionality available to users. They can be separated with feature flags, configuration, or progressive exposure. This distinction helps SRE because we can deploy safely without immediately exposing all customers to the change.

Specific pointers:

- Useful for canary releases and feature flags.
- Allows quick rollback of exposure without necessarily rebuilding the artifact.

3. What are your release gates?

Spoken answer: I use gates appropriate to risk: code quality, unit and integration tests, security scanning, artifact validation, infrastructure validation, deployment health, smoke tests and service-level checks. For higher-risk changes I also use approvals, canary validation and an explicit rollback decision point.

Specific pointers:

- Quality gate: test coverage/quality checks where defined.
- Security: dependency/container/IaC scans.
- Operational: capacity, dependencies, dashboards, alerts and rollback readiness.
- Reliability: error rate, latency and saturation after deployment.

4. How would you handle a failed production release?

Spoken answer: First I determine whether customers are impacted and stop further rollout. I compare the current release with the previous known-good version, check deployment events, application logs, metrics and recent infrastructure changes. If impact is significant and rollback is safe, I restore the last known-good version. Then I validate recovery through SLIs, document the incident timeline and create RCA actions so the same failure is less likely to recur.

Specific pointers:

- Avoid debugging for too long while impact is increasing.
- Preserve evidence before changing too much.
- Validate rollback dependencies, database compatibility and configuration compatibility.

5. How do you implement blue-green, canary and rolling deployments?

Spoken answer: Rolling deployment replaces instances gradually. Blue-green keeps two environments and switches traffic between them, which gives a fast rollback path but can cost more capacity. Canary sends a small percentage of traffic to the new version and increases exposure only when health and business indicators remain within thresholds. For reliability, I choose the strategy based on blast radius, statefulness, cost and rollback requirements.

Specific pointers:

- Canary is strong for high-risk application changes.
- Blue-green works well when parallel environments are feasible.
- Stateful/database changes require backward-compatible migration planning.

3. CI/CD Pipeline Integration & Tooling

1. How do you integrate tools into a CI/CD pipeline?

Spoken answer: I design the pipeline as a sequence of controlled stages where each tool produces an artifact or decision that the next stage consumes. For example, GitHub Actions can trigger on pull requests and merges; build tools compile/package; automated tests validate behavior; scanners check code, dependencies, containers or IaC; an artifact repository stores immutable packages; Terraform handles infrastructure; and deployment tooling promotes the same artifact through environments. The important part is traceability, quality gates and failure handling.

Specific pointers:

- Trigger → checkout → dependency install → lint/unit test → build → security/quality scans → package → publish artifact → deploy → smoke/integration tests → progressive rollout → post-deploy validation.
- Use secrets from a secure secret store, not repository files.
- Use reusable workflows/actions to standardize controls.
- Promote the same immutable artifact rather than rebuilding differently per environment.

Likely follow-up: How would you design a GitHub Actions pipeline for a microservice?

2. What is CI versus CD?

Spoken answer: Continuous Integration means integrating code frequently and validating it automatically through builds and tests. Continuous Delivery means keeping software in a releasable state through automated deployment preparation and controls. Continuous Deployment goes further by automatically releasing validated changes to production. The choice depends on organizational risk, controls and release maturity.

Specific pointers:

- Mention automated testing, artifact management, environment promotion and approvals.

3. How would you secure a CI/CD pipeline?

Spoken answer: I use least-privilege identities, short-lived credentials where possible, protected branches and environments, secret management, dependency and container scanning, IaC scanning, signed or immutable artifacts where supported, audit logs and controlled approvals. I also avoid printing secrets in pipeline logs and restrict who can

modify workflows.

Specific pointers:

- OIDC/federated identity can reduce long-lived cloud credentials.
- Separate build permissions from production deployment permissions.
- Review third-party actions and pin versions where appropriate.

4. How do you troubleshoot a pipeline that suddenly became slow?

Spoken answer: I compare the current execution with a known-good run and identify which stage increased in duration. Then I inspect runner capacity, dependency download time, test duration, artifact upload/download, external service latency and recent workflow changes. I use timestamps and stage-level metrics rather than guessing. Once the bottleneck is identified, I optimize caching, parallelization, runner capacity or the specific slow dependency.

Specific pointers:

- Measure before optimizing.
- Check external package registries and artifact repositories.
- Parallelize independent tests carefully.

4. SRE Metrics: SLI, SLO, SLA & Error Budget

Memorize this: SLI = what you measure. SLO = the reliability target. SLA = the external/business commitment, usually with consequences. Error budget = how much unreliability is permitted by the SLO.

Term	Meaning	Example	Interview distinction
SLI	A quantitative indicator of service behavior.	99.95% successful requests; p95 latency 280 ms.	Measurement.
SLO	Target for an SLI over a defined period.	99.9% availability per 30 days.	Internal reliability objective.
SLA	Business/customer commitment with defined terms and consequences.	99.9% monthly availability with service-credit terms.	External/business commitment.
Error Budget	Allowed unreliability implied by the SLO.	For 99.9%, about 43m 12s/month.	Risk/release capacity derived from SLO.

1. How do you define an SLI for an eCommerce API?

Spoken answer: I start from the user journey and business-critical endpoints. For an API, common SLIs are successful request ratio, latency and sometimes freshness or correctness depending on the service. I define the measurement carefully and validate it against real customer impact. The SLO should be achievable and meaningful enough to guide engineering decisions.

Specific pointers:

- Availability: successful eligible requests / total eligible requests.
- Latency: proportion of requests below a defined threshold or percentile-based objective.
- Consider checkout, payment, catalog and search separately if their reliability needs differ.

2. What happens when an SLO is breached?

Spoken answer: I first determine whether the breach is real and understand customer impact. Then I look at the error-budget state and whether recent changes contributed. A breach should trigger reliability work: incident response if active, RCA and corrective actions. Repeated budget consumption may require reducing release risk, improving capacity, fixing dependencies or changing the SLO if evidence shows it is incorrectly defined.

Specific pointers:

- Do not treat SLO as a dashboard-only metric.
- Tie SLOs to engineering priorities and release risk.

3. Explain error budgeting.

Spoken answer: If the SLO allows a small amount of unreliability, that allowance is the error budget. For example, a 99.9% monthly availability target permits roughly 0.1% unavailability. Teams can spend that budget on releases and controlled risk. If the budget is being exhausted too quickly, reliability work takes priority over additional risky change.

Specific pointers:

- Use budget burn rate rather than only total monthly consumption.
- Connect budget policy to release governance.

4. What is a burn rate?

Spoken answer: Burn rate tells us how quickly the service is consuming its error budget relative to the acceptable rate. A high burn rate means the service is heading toward an SLO violation quickly. It is useful for alerting because it can identify serious reliability degradation sooner than waiting for the monthly SLO to be breached.

Specific pointers:

- Fast-burn alerts detect severe incidents.

- Slow-burn alerts detect sustained degradation.

5. Production Operations & Incident Management

1. Walk me through your response to a P1 production incident.

Spoken answer: I start by confirming impact, affected users, scope and severity, then establish clear ownership and communication. I check recent changes and the main golden signals—traffic, errors, latency and saturation—then correlate logs and traces to isolate the failing component or dependency. My immediate goal is mitigation: rollback, failover, scaling, traffic control or disabling a problematic feature when appropriate. After recovery I verify SLIs, capture the timeline and complete RCA with corrective and preventive actions.

Specific pointers:

- Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent.
- Assign incident commander and technical/communications roles for major incidents.
- Use ServiceNow/ITSM records where required.
- Keep a clear incident timeline with timestamps and decisions.

2. How do you troubleshoot a microservice returning HTTP 500s?

Spoken answer: I first quantify the impact and whether failures are isolated to one endpoint, version, region or dependency. I check request rate, error rate and latency, then inspect application logs using a correlation or request ID. I compare healthy versus failing instances and check recent deployments, configuration, secrets, dependency health, database connections and resource saturation. If the issue is release-related, I evaluate rollback quickly. Otherwise I isolate the dependency or code path causing the failure and mitigate before deep RCA.

Specific pointers:

- Check ingress/API gateway → service → dependency chain.
- Compare pod/container restarts, CPU/memory, connection pools and downstream latency.
- Do not assume 500 means application code only.

3. How do you conduct an RCA?

Spoken answer: I build a factual timeline from monitoring, logs, deployment records and incident notes. I identify the technical and systemic contributing factors rather than stopping at the first symptom. I document customer impact, detection gap, mitigation, root cause, contributing conditions and actions with owners and due dates. The output should improve the system through tests, monitoring, automation, architecture or process.

Specific pointers:

- Separate root cause, contributing factors and detection/response gaps.
- Track actions to completion.

4. How do you reduce MTTR?

Spoken answer: I focus on both detection and recovery. Better actionable alerts and dashboards reduce MTTD; standardized runbooks, automation, safe rollback, good observability and clear ownership reduce recovery time. I also analyze repeated incidents and automate common mitigation steps. The goal is not only faster firefighting but fewer incidents over time.

Specific pointers:

- Measure MTTD, recovery time and recurring incident trends.
- Automate repetitive mitigations only after understanding failure modes.

6. Observability Practices

1. What does good observability mean to you?

Spoken answer: Observability is the ability to understand internal system behavior from externally exposed signals. I use metrics, logs and traces together. Metrics tell me that something changed and how broadly; logs provide event context; traces show request flow across distributed services. Good observability also includes meaningful dashboards, correlation IDs, actionable alerts and enough context to diagnose without manually searching hundreds of systems.

Specific pointers:

- Golden signals: latency, traffic, errors, saturation.
- Add business SLIs such as checkout success where appropriate.
- Use structured logs and consistent correlation/request IDs.
- Alerts should be actionable and tied to user or service impact.

2. How would you improve noisy monitoring?

Spoken answer: I classify alerts by actionability and customer impact. I remove duplicates, tune thresholds, use symptom-based alerts for important services and route alerts to the right ownership. I also use multi-window or burn-rate logic where appropriate so transient noise does not page the team unnecessarily. Every alert should have a clear action or investigation path.

Specific pointers:

- Track alert volume, false positives, acknowledgement time and recurring alerts.
- Dashboards are for diagnosis; alerts are for action.

3. What would you monitor for a Kubernetes application?

Spoken answer: At cluster level I monitor node capacity, CPU, memory, disk, network and control-plane health where available. At workload level I monitor pod restarts, pending pods, OOM kills, readiness/liveness failures, resource requests versus usage and deployment health. At application level I monitor request rate, errors, latency and saturation, plus business SLIs. I also correlate Kubernetes events, logs and traces during incidents.

Specific pointers:

- Managed Kubernetes: AKS/GKE concepts.
- Monitor HPA behavior and whether scaling is constrained by cluster or downstream capacity.
- Check service/ingress metrics for network-layer issues.

7. Terraform / Infrastructure as Code

1. How do you structure Terraform for production?

Spoken answer: I separate reusable modules from environment-specific configuration, keep Terraform in version control and use remote state with locking where supported. Changes go through review and CI validation, including formatting, validation, plan review and security checks. I keep secrets out of state and code where possible, use variables and outputs carefully, and control production apply permissions.

Specific pointers:

- Module boundaries should represent reusable infrastructure patterns.
- Remote state + locking prevents concurrent state corruption.
- Use terraform fmt, validate, plan and policy/security checks.
- Use separate state per environment or an appropriate state strategy.

2. What is Terraform state and why is it important?

Spoken answer: State maps Terraform configuration to real infrastructure and lets Terraform determine what must change. In production I prefer a remote backend with access control and locking. I protect state because it can contain sensitive values and because corruption or concurrent updates can cause serious problems.

Specific pointers:

- State is part of Terraform's reconciliation model.
- Never casually delete or manually edit state.

3. What happens if Terraform shows unexpected changes?

Spoken answer: I do not immediately apply. I inspect the plan, compare configuration and state, check provider version changes, recent manual changes and resource drift. I decide whether the resource needs to be imported/reconciled or whether the code should change. The key is understanding why the plan differs before making changes.

Specific pointers:

- Drift can come from manual changes, provider behavior, defaults or configuration changes.
- Review replacement versus in-place update carefully.

4. How do you manage Terraform in CI/CD?

Spoken answer: A pull request triggers formatting, validation, lint/security checks and a plan. The plan is reviewed and controlled. After approval, the protected deployment workflow applies the intended change using a controlled identity. Production permissions are separated from normal developer permissions, and the workflow records who approved and applied the change.

Specific pointers:

- Plan on PR; controlled apply after approval.
- Avoid unrestricted production terraform apply from developer laptops.

8. Scripting Basics – Python / Bash / PowerShell

1. What scripting tasks are common in SRE?

Spoken answer: Common tasks include log cleanup, disk and memory checks, process/service health checks, log parsing, API calls, housekeeping, backups, deployment helpers and incident diagnostics. I prefer scripts that are idempotent where possible, validate inputs, fail clearly, produce useful logs and return meaningful exit codes.

Specific pointers:

- Bash: grep, awk, sed, find, xargs, sort, uniq, df, du, ps, systemctl, curl.
- Python: requests, subprocess, pathlib, json, logging, argparse, exception handling.
- Schedule repetitive housekeeping with cron or platform-native automation.

2. A server filesystem is 95% full. What do you do?

Spoken answer: I confirm which filesystem is full and identify the top consumers. Then I check logs, temporary files, deleted-but-open files and application growth patterns. I free space safely using approved housekeeping or log rotation, never deleting active application data blindly. After stabilizing capacity, I identify why growth occurred and automate prevention such as rotation, retention or capacity alerts.

Specific pointers:

- df -h → du → find large files → lsof for deleted-open files → retention → prevention.
- Protect application/data directories.

3. What makes a production script reliable?

Spoken answer: It should be deterministic, idempotent where possible, validate assumptions, handle errors, use timeouts for external calls, log enough context without exposing secrets, return correct exit codes and have a safe failure mode. I also test it on non-production data or environments before automating it against production.

Specific pointers:

- Use set -euo pipefail carefully in Bash.
- Use bounded retries for transient operations.

- Never hard-code credentials.

9. Application Scaling & Performance Engineering

1. How would you scale an application from 100 to 100,000 concurrent users?

Spoken answer: I would not scale only by increasing server size. First I establish workload characteristics and bottlenecks through load testing and observability. Then I scale horizontally behind a load balancer, make application instances stateless where possible, introduce caching, optimize database access and connection pools, use asynchronous queues for non-critical work and configure Kubernetes HPA/cluster autoscaling. I also verify downstream capacity because scaling the frontend can overload the database or third-party APIs.

Specific pointers:

- Measure CPU, memory, latency, RPS, DB connections, cache hit rate and queue depth.
- Use HPA for workload demand and cluster autoscaling for node capacity.
- Protect dependencies with rate limits, circuit breakers and bounded concurrency.
- Load test realistic traffic patterns before production scale changes.

2. How do you identify an application performance bottleneck?

Spoken answer: I start with user-facing latency and break it down by service and dependency. I use traces to identify slow spans, metrics to quantify resource saturation and logs for specific errors or timeouts. I then validate hypotheses with profiling or controlled tests. Typical bottlenecks include inefficient queries, connection pools, external API latency, CPU-intensive code, garbage collection, lock contention or network saturation.

Specific pointers:

- Do not optimize based only on CPU.
- Use p50/p95/p99 latency appropriately.
- Correlate performance with traffic and deployment changes.

10. Resilience Engineering

1. What is resilience in a distributed application?

Spoken answer: Resilience is the ability to continue providing acceptable service despite failures or degradation. I design for failure using redundancy, health checks, timeouts, bounded retries with exponential backoff and jitter, circuit breakers, bulkheads, graceful degradation and automated recovery. I also validate resilience through testing and failure scenarios rather than assuming replicas alone make a system resilient.

Specific pointers:

- Avoid retry storms.
- Use readiness probes to control traffic and liveness probes carefully.
- Define RTO/RPO where applicable.
- Consider dependency and regional failure, not just pod failure.

2. Why are timeouts and retries important?

Spoken answer: Without timeouts, a slow dependency can consume threads or connections until the service becomes unavailable. Retries can recover transient failures, but uncontrolled retries can amplify an outage. I use bounded retries, exponential backoff and jitter, and only retry operations that are safe to retry. I also consider end-to-end timeout budgets.

Specific pointers:

- Retry only transient failures.
- Idempotency matters for writes.
- Circuit breakers can stop repeated calls to an unhealthy dependency.

3. How would you make a checkout service resilient?

Spoken answer: I would identify critical dependencies such as catalog, inventory, payment and order services and define what can fail independently. Payment and order operations need idempotency. I would use timeouts, bounded retries, circuit breakers and durable messaging where asynchronous processing is acceptable. I would monitor checkout success as a business SLI and design graceful degradation for non-critical features.

Specific pointers:

- Protect payment calls from duplicate processing.
- Use correlation IDs across the transaction.
- Prioritize the critical path over secondary features.

11. Kubernetes / AKS / GKE Troubleshooting

1. A pod is in CrashLoopBackOff. What do you check?

Spoken answer: I first check pod status, recent events and container logs, including previous-container logs. Then I inspect the pod spec, environment variables, ConfigMaps and Secrets, probes, resource limits, image/version and recent deployment changes. I determine whether it is an application startup failure, configuration issue, dependency failure or resource problem.

Specific pointers:

- kubectl get pods
- kubectl describe pod
- kubectl logs --previous
- kubectl get events --sort-by=.lastTimestamp
- Check OOMKilled, probe failures and missing config/secrets.

2. A pod is Pending. What is your troubleshooting sequence?

Spoken answer: I inspect events first because the scheduler usually explains why placement failed. I check node capacity, resource requests, taints/tolerations, node selectors, affinity rules, PVC availability and quotas. I distinguish insufficient capacity from scheduling constraints and then verify cluster autoscaling if it should add capacity.

Specific pointers:

- kubectl describe pod
- kubectl get nodes
- Check PVC/StorageClass if storage is involved.
- Check resource requests rather than only current usage.

3. How do readiness and liveness probes differ?

Spoken answer: Readiness determines whether the pod should receive traffic. Liveness determines whether the container is unhealthy enough to restart. A common mistake is an overly aggressive liveness probe that restarts an application during temporary startup or dependency conditions. Readiness should protect traffic routing; liveness should detect a genuinely stuck process.

Specific pointers:

- Startup probes can protect slow-starting applications.
- Avoid making liveness depend on every downstream service without a strong reason.

4. How do you troubleshoot an API that works inside the pod but not externally?

Spoken answer: I trace the network path: application listening address/port, Service selector and endpoints, service port versus targetPort, NetworkPolicy, ingress or API gateway, DNS and load balancer. I compare internal service access with external access and inspect ingress/controller logs. This narrows the issue from application to service discovery to edge routing.

Specific pointers:

- kubectl get svc,endpoints
- kubectl describe ingress
- Check DNS resolution and TLS.
- Validate selectors and ports.

12. ITIL / ServiceNow / Cross-functional Operations

1. How do SRE and ITIL work together?

Spoken answer: I see them as complementary. ITIL provides structured service management processes such as incident, problem, change and service request management. SRE adds engineering practices such as SLOs, error budgets, automation and reliability measurement. In production I can follow change controls while still using automation and measurable reliability outcomes.

Specific pointers:

- Incident = restore service.
- Problem = address underlying recurring causes.
- Change = controlled modification with risk and approval.
- Use ServiceNow or equivalent ITSM records for traceability.

2. How do you handle a change that caused an incident?

Spoken answer: I correlate the incident timeline with the change and verify evidence before assigning causality. If the change is the likely cause and rollback is safe, I prioritize restoration. I record the change and incident relationships in the ITSM system, communicate impact and later review whether the change process or automated validation should be improved.

Specific pointers:

- Do not assume correlation is causation without evidence.
- Use deployment metadata in observability to make correlation easier.

3. How do you work in a cross-functional product team?

Spoken answer: I try to make reliability a shared product concern rather than an SRE-only responsibility. I work with developers, QA, product, security and platform teams on service indicators, release readiness, incident learnings and reliability improvements. I make operational risks visible through metrics and clear actions, and automate repetitive work so the team can spend more time on engineering improvements.

Specific pointers:

- This directly matches the JD's reliability-as-expertise and capability-sharing model.
- Bring reliability requirements into design and delivery, not only production support.

13. DevOn JD-Specific Rapid-Fire Questions

1. What KPIs would you report to product leadership?

Spoken answer: Availability/SLO attainment, error-budget consumption, incident count and severity, MTTD/MTTR, change failure rate, deployment frequency, latency, capacity/saturation and recurring incident trends.

2. How do you make monitoring preventive rather than reactive?

Spoken answer: I identify leading indicators such as capacity growth, error-budget burn, queue depth, latency trends, certificate expiry, disk growth and dependency health. I automate housekeeping and remediation where safe and alert before user-visible failure.

3. What is change failure rate?

Spoken answer: The percentage of deployments that cause a failure requiring remediation, rollback, hotfix or incident. It is useful alongside deployment frequency because deployment volume alone does not indicate delivery health.

4. What is toil?

Spoken answer: Manual, repetitive, automatable operational work that scales with system size. I measure recurring toil and automate it when the cost and risk justify the investment.

5. What makes an alert actionable?

Spoken answer: It indicates a condition requiring human or automated action, has a clear owner and runbook, has an appropriate threshold or symptom, and avoids paging for transient noise.

6. Why use structured logging?

Spoken answer: It makes logs machine-searchable and consistent. Fields such as timestamp, service, environment, severity, request ID, trace ID and error code make correlation and automated analysis easier.

7. What is graceful degradation?

Spoken answer: When a dependency or non-critical capability fails, the service continues providing the most important functionality instead of failing completely.

8. What is a runbook?

Spoken answer: A documented operational procedure for diagnosing or mitigating a known condition. Good runbooks contain symptoms, checks, commands, decision points and safe recovery actions.

9. What is a postmortem?

Spoken answer: A factual review after an incident covering impact, timeline, root and contributing causes, response effectiveness and concrete actions to reduce recurrence.

10. Why are business SLIs important?

Spoken answer: Infrastructure can look healthy while users are failing. Business-oriented SLIs such as checkout success or order completion connect reliability engineering to actual customer outcomes.

14. Scenario-Based Questions – Practice These Out Loud

1. Checkout success drops from 99.8% to 94% immediately after a release. What do you do?

Spoken answer: Establish impact and scope, stop further rollout, compare the new version against the previous version, inspect checkout traces and logs, verify payment/inventory dependencies, and check the release diff. If evidence points to the release and rollback is safe, roll back and validate recovery using the checkout SLI. Then complete RCA and add a regression test or release gate.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

2. API latency is normal at p50 but p99 is suddenly high. What does that tell you?

Spoken answer: The average experience can hide tail latency. I would investigate a subset of slow requests, often caused by specific endpoints, dependencies, database queries, connection pool exhaustion, GC, noisy neighbors or retries. Traces and high-cardinality dimensions help isolate the affected path.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

3. Kubernetes HPA is scaling pods, but latency keeps increasing. Why?

Spoken answer: The bottleneck may be downstream or cluster-level. I would check node capacity, scheduling latency, database connections, cache hit rate, external dependencies, network saturation and whether new pods are actually ready and receiving traffic. Scaling the application tier does not solve a constrained database or third-party API.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

4. Terraform plan wants to replace a production resource. What do you do?

Spoken answer: Do not apply blindly. Identify exactly which attribute forces replacement, compare state/configuration/provider behavior, check for drift and determine business impact. If unintended, fix configuration or state reconciliation. If intended, plan migration with backup, dependency analysis, maintenance strategy and recovery considerations.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

5. A disk fills every night due to logs. How do you solve it permanently?

Spoken answer: First stabilize the current incident safely. Then determine which process generates the logs and why retention is missing or insufficient. Implement log rotation/retention, centralize logs where appropriate, monitor disk growth, alert before capacity becomes critical and test the housekeeping job.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

6. A third-party payment API becomes slow. How do you prevent cascading failure?

Spoken answer: Set strict timeouts, bounded retries with backoff and jitter, circuit breaking, concurrency limits and clear fallback behavior. Avoid retrying non-idempotent operations blindly. Monitor dependency latency and failure rate separately and define the customer experience when payment is temporarily unavailable.

Specific pointers:

- State the customer impact first.
- Use evidence from metrics/logs/traces.
- Mitigate before deep diagnosis if impact is growing.
- End with a prevention/control improvement.

15. Practical Command Cheat Sheet

Linux / Bash

```
# Disk and largest files
df -h
du -xh /var/log | sort -h | tail
find /var/log -type f -size +500M -ls

# CPU / memory / processes
uptime
top
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head
free -m

# Logs / HTTP / network
tail -f /var/log/app.log
grep -i "error" app.log | tail
curl -v https://service.example.com/health
ss -lntp
dig service.example.com

# Service
systemctl status <service>
journalctl -u <service> --since "30 min ago"
```

Kubernetes

```
kubectl get pods -A
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get svc,endpoints
kubectl describe ingress <ingress>
kubectl top pods
kubectl top nodes
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
```

Terraform

```
terraform fmt -check
terraform init
terraform validate
terraform plan
terraform apply
terraform state list
terraform show
```

Interview caution: Commands are examples; adapt them to the actual environment and organization's operational controls. Never run destructive commands in production without authorization and impact assessment.

16. 30-Minute Final Revision Plan

0–5 min	Memorize SLI vs SLO vs SLA and the error-budget relationship.
5–10 min	Practice the P1 incident answer and one microservice 500 troubleshooting scenario.
10–15 min	Practice GitHub Actions pipeline stages and release/rollback strategy.
15–20 min	Practice Terraform production structure, state, drift and CI/CD integration.
20–25 min	Practice observability, Kubernetes Pending/CrashLoopBackOff and scaling questions.
25–30 min	Practice one eCommerce checkout scenario and one resilience/dependency-failure scenario.

Final interview positioning: Reliability is not only monitoring production. It is designing measurable reliability targets, controlling change, automating operational work, detecting problems early, restoring service quickly, learning from incidents and continuously reducing the probability and impact of failure.

Integrated Source-Question Appendix

This appendix consolidates the uploaded DevOn/SearchStax interview materials into the existing DevOn SRE PDF. It contains 238 unique interviewer questions/scenarios after removing duplicate question text across the uploaded files.

Source handling: The questions and answers below are extracted from the uploaded handbooks. They are retained as source-derived material rather than silently rewritten into generic answers. Where a source uses a specific product, command, metric or scenario, that terminology is preserved.

DevOn / SearchStax Round-2 Handbook	102
SearchStax Senior SRE Interview Guide	59
SRE Interview Recall Handbook	77
Total	**238**

DevOn / SearchStax Round-2 Handbook

k8s

Round-2 · k8s

1. A pod is stuck in Pending. How do you troubleshoot it?

Source answer:
Pending means the scheduler has not been able to bind the pod to any node yet — the container hasn't even started.

- Start with `kubectl describe pod <pod>` and read the **Events** section at the bottom — it almost always states the exact scheduling failure, e.g. "0/5 nodes are available: 3 Insufficient cpu, 2 node(s) had taint {dedicated: true}".
- Common causes to check in order:
- **Insufficient resources** — requested CPU/memory exceeds what any node has free. Check with `kubectl top nodes` and `kubectl describe nodes`.
- **nodeSelector / affinity mismatch** — pod requires labels no node has.
- **Taints without tolerations** — node has a taint the pod doesn't tolerate.
- **Unbound PVC** — pod references a PVC that hasn't bound to a PV yet.
- **Pod count limit** — node already at max-pods.
- **Topology spread constraints** — can't satisfy the requested spread across zones/nodes.
- Fix depends on the cause: scale the node group, loosen the selector/affinity, add a toleration, resolve the PVC, or relax the spread constraint.

Round-2 · k8s

2. How do you debug a CrashLoopBackOff?

Source answer:
CrashLoopBackOff means the container starts, exits, and kubelet keeps restarting it with an exponential backoff delay — it's a symptom, not a root cause.

- `kubectl describe pod <pod>` — check **Restart Count** and **Last State: Terminated → Reason** (e.g. `OOMKilled`, `Error`) and **Exit Code**:
- exit code `137` = SIGKILL, usually an OOM kill or liveness probe kill
- exit code `1` = generic application error
- exit code `143` = SIGTERM (graceful shutdown signal received)
- `kubectl logs <pod> --previous` — this is the key command; it shows logs from the **crashed** instance, not the new one that just started.
- If logs are empty, check the container's actual command/entrypoint — a typo there exits immediately with nothing logged.
- If it's OOMKilled, check `kubectl top pod` history in Grafana and either raise the memory limit or fix a leak.
- If it's a failing **liveness probe** causing the restart (not an app crash at all), the fix is tuning `initialDelaySeconds` / `periodSeconds` / `failureThreshold`, not the app.

Round-2 · k8s

3. Pod shows ImagePullBackOff or ErrImagePull — how do you fix it?

Source answer:
The kubelet can't pull the container image.

- `kubectl describe pod <pod>` → Events will show one of:
- "manifest not found" → wrong image name or tag
- "unauthorized" / "authentication required" → missing or wrong `imagePullSecrets` for a private registry
- timeout / DNS errors → node can't reach the registry (network/DNS/firewall)
- "toomanyrequests" → registry rate limiting (common with public Docker Hub)
- Checklist:
- Confirm the image and tag actually exist: `docker pull <image>` from a machine with the same network path, or check the registry UI.
- Confirm the pod's `imagePullSecrets` exists in the same namespace and has valid credentials: `kubectl get secret <name> -o yaml`.
- Confirm the node can resolve/reach the registry (`curl`/`nslookup` from the node or a debug pod).
- For rate limiting, use an authenticated pull or mirror the image to a private registry.

Round-2 · k8s

4. What causes CreateContainerConfigError and how do you resolve it?

Source answer:
This means the container spec references a ConfigMap or Secret (or a key inside one) that doesn't exist.

- `kubectl describe pod <pod>` → Events typically say something like "configmap \"app-config\" not found" or "couldn't find key API_KEY in Secret app-secrets".
- Fix:
- Confirm the ConfigMap/Secret exists in the **same namespace** as the pod — this is the single most common mistake.
- Confirm the exact key name matches what's referenced in `env.valueFrom` or `envFrom`.
- If it was just created, make sure the pod was recreated afterward — existing pods don't pick up a newly-created ConfigMap automatically unless it's mounted as a volume and the app reloads it.

Round-2 · k8s

5. A pod keeps getting OOMKilled. Walk through your diagnosis.

Source answer:
OOMKilled means the container exceeded its memory **limit** and the kernel's cgroup OOM killer terminated it — Kubernetes reports this as `Reason: OOMKilled`, exit code 137.

- Confirm it in `kubectl describe pod` under **Last State**.
- Pull up `container_memory_working_set_bytes` for that pod in Grafana over the last few hours/days:
- A **sawtooth pattern** that resets on restart usually means the limit is simply too low for legitimate peak usage.
- A **steady upward slope that never comes back down**, even across low-traffic periods, points to an actual memory leak in the app.
- Cross-check `kubectl top pod` for current usage vs the configured limit.
- Resolution:
- If it's sizing: raise `resources.limits.memory` based on observed peaks plus headroom, and make sure `requests` are set sensibly too (a request far below the real usage causes bad scheduling density).
- If it's a leak: this becomes an app-level investigation (heap dump/profiler — see the memory-leak scenario question) rather than a Kubernetes fix.
- For JVM apps specifically, remember the JVM's own heap settings need to fit inside the container memory limit, or the JVM itself can be killed even though it thinks it has room.

Round-2 · k8s

6. What does an Evicted pod status mean and how do you fix it?

Source answer:
Eviction happens when the **node** — not the container — is under resource pressure, and kubelet proactively kills pods to reclaim resources (this is different from OOMKilled, which is a per-container cgroup limit being hit).

- `kubectl describe node <node>` → check **Conditions** for `MemoryPressure`, `DiskPressure`, or `PIDPressure` = True.
- `kubectl get pod` shows `STATUS: Evicted` with a message explaining which resource was under pressure.
- Common root causes: too many pods scheduled onto the node without proper requests (overcommit), a runaway process filling disk (logs, temp files), or genuinely undersized nodes.
- Fixes:
- Set accurate `requests`/`limits` on all workloads so the scheduler doesn't overcommit a node.
- Clean up disk pressure sources (log rotation, orphaned images — `crictl rmi --prune` or similar).
- Consider PriorityClasses so critical pods are evicted last.
- Add node capacity or enable cluster autoscaling.

Round-2 · k8s

7. You see a pod in Completed state that you didn't expect — what's going on?

Source answer:
`Completed` means every container in the pod exited with code 0 — this is expected and normal for **Jobs**, but unexpected on a pod that's supposed to be a long-running service (like one backed by a Deployment).

- Check `restartPolicy` on the pod spec — for a Deployment-managed pod it should be `Always`.
- Check the container's actual entrypoint/command — if the main process exits after finishing its work (e.g. a script that runs once and returns) instead of blocking/serving, Kubernetes will faithfully report Completed even though you intended a long-running server.
- Check `kubectl logs <pod>` for the last few lines to see why the process returned control instead of continuing to run.
- Fix is almost always at the application/entrypoint level — make sure the foreground process is the actual server (common Dockerfile mistake: launching the server in the background and letting the entrypoint script exit).

Round-2 · k8s

8. How do you troubleshoot an init container that keeps failing?

Source answer:
Init containers run sequentially, in order, before any main container starts — if one fails, the pod stays stuck (shown as `Init:Error` or `Init:CrashLoopBackOff`) and the app container never even starts.

- `kubectl describe pod <pod>` shows which init container index is failing.
- `kubectl logs <pod> -c <init-container-name>` — you must specify the init container name explicitly, `kubectl logs` alone won't show it.
- Typical causes: a dependency-wait script (waiting for a DB/service to become reachable) that never succeeds, a migration script erroring out, or a permissions issue on a mounted volume it's trying to prepare.
- Fix the underlying dependency or script logic; a good pattern is giving wait-for-dependency init containers a bounded retry loop with clear logging rather than an infinite silent loop.

Round-2 · k8s

9. Explain Kubernetes architecture at a high level.

Source answer:
Two halves: the **control plane** (the brain) and **worker nodes** (where workloads actually run).

- Control plane components:
- **kube-apiserver** — the front door; all communication (kubectl, controllers, kubelets) goes through it via REST, with authentication, authorization, and admission control.
- **etcd** — the cluster's single source of truth, a distributed key-value store holding all object state.
- **kube-scheduler** — watches for unscheduled pods and assigns them to a node based on resource requests, affinity, taints, etc.
- **kube-controller-manager** — runs the reconciliation control loops (Deployment, ReplicaSet, Node, and others) that continuously drive actual state toward desired state.
- **cloud-controller-manager** — cloud-provider-specific integration (load balancers, node lifecycle, routes).
- Worker node components:
- **kubelet** — the node agent; registers the node, watches for pods assigned to it, and drives the container runtime.
- **kube-proxy** — implements Service networking (iptables/IPVS rules).
- **container runtime** — via the CRI (containerd, CRI-O) actually runs containers.
- The core idea to emphasize in an interview: everything is a **declarative, level-triggered control loop** — controllers constantly reconcile observed state to desired state, not a one-shot imperative action.

Round-2 · k8s

10. Walk me through how a request actually reaches a pod.

Source answer:
- Client hits an external IP — usually a cloud Load Balancer in front of an Ingress controller (or directly a Service of type LoadBalancer).
- The cloud LB forwards to a **NodePort** on one or more nodes, which routes to the Ingress controller's pods.
- The **Ingress controller** (e.g. nginx) reads Ingress resources for host/path rules and proxies the request to the matching backend **Service**'s ClusterIP.
- **kube-proxy** on that node has already programmed iptables (or IPVS) DNAT rules from watching the Service's **Endpoints/EndpointSlice** — these rules load-balance the ClusterIP across the healthy backing pod IPs.
- The packet is rewritten to a specific pod IP and container port, and the pod's application handles it.
- Key detail interviewers listen for: only pods that are **Ready** appear in Endpoints — a failing readiness probe removes a pod from this path without killing it.

Round-2 · k8s

11. What actually happens, step by step, when a pod's container crashes?

Source answer:
- The container process exits; the container runtime reports this to kubelet.
- kubelet checks the pod's `restartPolicy`:
- `Always` (default for Deployments) → restart in place, with exponential backoff if it keeps happening (this is what produces CrashLoopBackOff).
- `OnFailure` → restart only on non-zero exit.
- `Never` → leave it terminated (typical for one-shot Jobs).
- Critically, restarting the **container** does not change the **pod's** identity or IP — it's still the same pod object, same node.
- Meanwhile, if the container is failing its **readiness** probe during this window, the Endpoints controller removes the pod from the Service's backend list so no traffic is routed to it.
- If the whole **pod** is deleted (not just a container restarting) — e.g. node failure — the ReplicaSet controller notices actual replica count is below desired and schedules a brand-new pod (new name, new IP) elsewhere.

Round-2 · k8s

12. Explain what kubelet does.

Source answer:
kubelet is the primary **node agent** — it's the thing that turns API server intent into running containers on that specific node.

- Registers the node with the API server and reports node status/capacity.
- Watches the API server for pods assigned (`spec.nodeName`) to its node.
- Calls the container runtime via the **CRI** to pull images and start/stop containers to match the pod spec.
- Executes **liveness/readiness/startup probes** and reports/acts on the results.
- Manages volume mounting via the **CSI**.
- Enforces resource requests/limits via cgroups.
- Reports pod and node status back to the API server so the rest of the control plane can reconcile against reality.

Round-2 · k8s

13. Explain etcd and why it matters operationally.

Source answer:
etcd is a distributed, strongly-consistent key-value store using the **Raft** consensus algorithm — it holds the entire cluster state (every object you'd see with `kubectl get`).

- Only the **kube-apiserver** talks to etcd directly; nothing else should.
- Needs quorum to accept writes, so it's run with an odd number of members (typically 3 or 5) for HA — losing quorum makes the cluster read-only/unresponsive to changes even if pods keep running.
- Default ports: **2379** (client) and **2380** (peer).
- It's latency-sensitive — recommend fast local SSD/NVMe storage, since disk fsync latency directly impacts write throughput and can destabilize the whole control plane.
- Operational musts: regular `etcdctl snapshot save` backups, monitoring db size against its quota (default ~2GB, can cause "mvcc: database space exceeded" errors), and periodic defragmentation.

Round-2 · k8s

14. Deployment vs StatefulSet — when do you use each?

Source answer:
Deployment — for stateless workloads:
- Pods are interchangeable, random name suffixes, no stable identity.
- No inherent stable storage per pod (though a PVC *can* be shared, it's not per-instance by design).
- Rolling updates are fast and don't need ordering.

StatefulSet — for stateful workloads (databases, Kafka, Zookeeper, Elasticsearch):
- Stable, predictable pod names (`app-0`, `app-1`, `app-2`, ...).
- Stable network identity via a headless Service, so each pod gets its own resolvable DNS name.
- Stable per-pod storage via `volumeClaimTemplates` — pod-1 always reattaches to its own PVC even after rescheduling.
- Ordered, sequential pod creation/scaling/deletion by default (configurable via `podManagementPolicy: Parallel` if you don't need that).

Interview one-liner: StatefulSet exists because Deployments explicitly assume pods are disposable and interchangeable — the moment "which specific instance am I talking to" matters, you need a StatefulSet.

Round-2 · k8s

15. DaemonSet vs Deployment — what's the difference and when would you pick a DaemonSet?

Source answer:
DaemonSet guarantees exactly one matching pod runs on every node (or every node matching a selector/toleration) — it's node-scoped, not replica-count-scoped.
- Typical uses: log shippers (Fluentd/Fluent Bit), node monitoring agents (node-exporter), CNI plugins, and kube-proxy itself often runs this way.
- As nodes are added/removed from the cluster, DaemonSet pods automatically appear/disappear — you don't set a replica count.

Deployment manages a fixed **replica count** regardless of how many nodes exist, and the scheduler places those replicas based on normal scheduling rules (which could put multiple replicas on the same node, or none on a given node).

Use a DaemonSet specifically when the workload's purpose is "one instance per node" infrastructure — anything else is a Deployment (or StatefulSet).

Round-2 · k8s

16. How does the Horizontal Pod Autoscaler (HPA) actually work?

Source answer:
- HPA periodically (default sync period ~15s) queries metrics for the target resource — CPU/memory via **metrics-server**, or custom/external metrics via something like the Prometheus Adapter.
- It computes desired replicas with roughly:
```
desiredReplicas = ceil( currentReplicas * ( currentMetricValue / targetMetricValue ) )
```
- It respects `minReplicas`/`maxReplicas` bounds and applies a **stabilization window** (scale-down default ~5 minutes) to avoid flapping — this is why HPA can feel "slow" to scale down even after load drops, which is by design.
- Common troubleshooting: HPA showing `<unknown>` for a metric usually means metrics-server (or the custom metrics API) isn't reachable, or the pods don't have resource **requests** set (HPA's CPU% target is relative to the request, so no request = no percentage to compute against).

Round-2 · k8s

17. Explain readiness vs liveness vs startup probes — and describe a real scenario where getting this wrong causes an outage.

Source answer:
- **Liveness probe** — "is this container alive/healthy?" Failing it → kubelet **restarts** the container.
- **Readiness probe** — "is this container ready to serve traffic right now?" Failing it → pod is **removed from Service Endpoints** (no traffic sent) but the container is *not* restarted.
- **Startup probe** — used for slow-starting apps; liveness/readiness checks are held off until the startup probe succeeds, so a legitimately slow boot doesn't get killed prematurely by an impatient liveness probe.

Real outage scenario: a new pod comes up during a deploy and starts receiving traffic (and 502s/500s) before its dependencies (DB pool, cache warm-up) are actually ready, because there's no readiness probe at all — or `initialDelaySeconds` is too short. The fix is a readiness probe that genuinely reflects "can I serve a real request right now," tuned with sensible `initialDelaySeconds`/`periodSeconds`/`failureThreshold`, and a startup probe for anything with a slow, variable boot time.

Round-2 · k8s

18. A PVC won't bind. How do you troubleshoot it?

Source answer:
- `kubectl describe pvc <name>` — the Events section tells you almost everything: "no persistent volumes available for this claim", a provisioning error from the CSI driver, or nothing at all (stuck Pending silently).
- Checklist:
- Does the referenced `storageClassName` actually exist? `kubectl get storageclass`.
- Is the StorageClass's provisioner (CSI driver) actually healthy — check its controller pods in `kube-system` or wherever it's deployed.
- Do `accessModes` and requested `size` match what's available (for static provisioning) or what the StorageClass supports?
- For **topology-aware** provisioning (common in cloud), is there a zone mismatch — the PV got provisioned in a zone the pod isn't scheduled to, or vice versa (`volumeBindingMode: WaitForFirstConsumer` exists specifically to avoid this by delaying binding until a pod is scheduled).
- Quota — check if a `ResourceQuota` on storage requests in the namespace is blocking it.

Round-2 · k8s

19. A pod is stuck Terminating and won't go away. What do you do?

Source answer:
- Check `kubectl get pod <pod> -o yaml` for a `deletionTimestamp` (deletion was requested) combined with non-empty `finalizers` — the pod can't be fully removed until whatever owns that finalizer clears it (common with CSI volume-attachment finalizers or custom operators).
- Check whether the container is ignoring `SIGTERM` — Kubernetes waits `terminationGracePeriodSeconds` (default 30s) then sends `SIGKILL`; if the app doesn't handle SIGTERM at all, you'll often see it hang right up to that grace period every time.
- Check the **node** the pod is on — if the node itself is `NotReady` (crashed, network-partitioned), kubelet isn't there to confirm the pod actually stopped, so it can hang indefinitely.
- Last resort: `kubectl delete pod <pod> --grace-period=0 --force` — but understand this doesn't guarantee the container process actually stopped if the node is unreachable; it just removes the API object, which can risk two copies of "the same" workload running if the node comes back.

Round-2 · k8s

20. kubectl logs shows nothing for a pod that's clearly having problems — what now?

Source answer:
- Confirm you're targeting the right container: `kubectl logs <pod> -c <container-name>` — in a multi-container pod (app + sidecar), the default without `-c` may not be the one you expect.
- Try `--previous` if the container already restarted — the "current" instance may genuinely have no logs yet.
- Consider that the app might be logging to a **file** instead of stdout/stderr — if so, `kubectl logs` will never show it; you need `kubectl exec` in (if the container is even up long enough) or a sidecar/log-shipper reading that file.
- If the container never got far enough to log anything (e.g. immediate exec format error, bad entrypoint), check `kubectl describe pod` events and the exit code rather than logs.
- For images with no shell (distroless), you can't `exec` a shell in — use an **ephemeral debug container**: `kubectl debug -it <pod> --image=busybox --target=<container>`, which attaches a debug container sharing the pod's namespaces without modifying the original spec.

Round-2 · k8s

21. kubectl describe shows a pod was rejected — Events mention a ResourceQuota or LimitRange. What's happening?

Source answer:
- A **ResourceQuota** caps total resource consumption (CPU, memory, object counts) per namespace — if creating this pod would push the namespace over quota, the API server rejects it at admission time. `kubectl describe resourcequota -n <ns>` shows current usage vs the hard limits.
- A **LimitRange** sets default/min/max requests and limits for containers in a namespace — if a pod doesn't specify requests/limits and none of the LimitRange defaults satisfy a constraint (e.g. below the enforced minimum), creation fails.
- `kubectl get events -n <ns>` will show something like "exceeded quota: compute-quota, requested: ..., used: ..., limited: ...".
- Fix: either raise the quota (a capacity-planning conversation, not just a kubectl command), reduce the workload's requested resources, or explicitly set requests/limits that satisfy the LimitRange.

Round-2 · k8s

22. A node shows NotReady. How do you troubleshoot it?

Source answer:
- `kubectl describe node <node>` — check **Conditions**: `Ready=False/Unknown`, and look at `MemoryPressure`/`DiskPressure`/`PIDPressure`/`NetworkUnavailable`.
- `NotReady` fundamentally means kubelet has stopped reporting heartbeats to the API server within the expected window — causes include: kubelet process crashed, the node lost network connectivity to the control plane, the node ran out of disk/PIDs so badly kubelet itself can't function, or a certificate expired (kubelet client cert rotation failures are a classic silent killer).
- If you have node access: `journalctl -u kubelet -f` for the actual error, `systemctl status kubelet`.
- Kubernetes' own reaction: after a node stays NotReady past the pod-eviction-timeout, the control plane marks its pods for eviction/rescheduling elsewhere (assuming capacity exists) — so a NotReady node is also indirectly a capacity/scheduling problem for the rest of the cluster.

Round-2 · k8s

23. A rolling update is stuck and not progressing. What do you check?

Source answer:
- `kubectl rollout status deployment/<name>` — tells you directly if it's waiting.
- `kubectl describe deployment <name>` → **Conditions** — look for `ProgressDeadlineExceeded`, which means new pods aren't becoming Ready within `progressDeadlineSeconds`.
- Check the **new** ReplicaSet's pods specifically (`kubectl get rs`, then `kubectl describe pod` on one of the new ones) — a rollout can't proceed past `maxUnavailable`/`maxSurge` until enough new pods pass readiness, so this is almost always really "why won't the new pods become Ready" (bad image tag, failing readiness probe, missing config) wearing a rollout-shaped disguise.
- If it's genuinely broken and needs to stop bleeding now: `kubectl rollout undo deployment/<name>` rolls back to the previous ReplicaSet immediately.

net

Round-2 · net

24. Explain ClusterIP, NodePort, LoadBalancer, and Ingress — when do you use each?

Source answer:
- **ClusterIP** (default) — a stable virtual IP reachable only inside the cluster. Use for internal service-to-service traffic.
- **NodePort** — exposes the Service on a static port (default range **30000-32767**) on every node's IP. Rarely used directly in production; mostly a building block for LoadBalancer/Ingress.
- **LoadBalancer** — asks the cloud provider to provision an external load balancer that typically targets the Service's NodePort. One LB per Service = can get expensive/hard to manage at scale.
- **Ingress** — L7 HTTP/HTTPS routing (host- and path-based) handled by an Ingress **controller** (e.g. nginx, Traefik) that itself usually sits behind a single LoadBalancer — so you get one external IP fronting many Services instead of one LB each. This is the standard way to expose multiple HTTP apps cost-effectively.

Round-2 · net

25. Explain CoreDNS and its role in the cluster.

Source answer:
CoreDNS is the cluster's DNS server (replaced the older kube-dns), typically run as a Deployment (commonly 2 replicas) in `kube-system`, fronted by its own Service.
- Every pod's `/etc/resolv.conf` is configured to point at CoreDNS's ClusterIP.
- It resolves Service DNS names in the form `<service>.<namespace>.svc.cluster.local` (and shorter forms within the same namespace/search domain).
- Configuration lives in the `Corefile` (a ConfigMap) — plugins include the `kubernetes` plugin (serves cluster records) and `forward` (sends non-cluster queries upstream), plus caching.
- It's a common single point of friction — if CoreDNS is unhealthy, seemingly unrelated services start failing intermittently with connection errors that are actually failed name resolution.

Round-2 · net

26. How do you debug DNS resolution issues inside the cluster?

Source answer:
- Exec into an existing pod (or spin up a throwaway one) and test directly: `kubectl run -it --rm dnsutils --image=gnzsnz/dnsutils -- nslookup kubernetes.default`.
- Check the pod's `/etc/resolv.conf` — pay attention to `ndots` (default 5 in Kubernetes), which causes even fully-qualified external lookups to first try several internal search-domain suffixes, adding latency and log noise.
- Check CoreDNS pods themselves: `kubectl get pods -n kube-system -l k8s-app=kube-dns`, `kubectl logs` on them for `SERVFAIL`/errors, and `kubectl top pod` in case they're CPU-throttled and dropping queries under load.
- Check the CoreDNS ConfigMap (`Corefile`) for a `forward` misconfiguration if external lookups fail but internal ones work.
- Confirm no `NetworkPolicy` is blocking UDP/TCP port 53 to `kube-system` — this is an easy one to miss once NetworkPolicies are introduced.

Round-2 · net

27. How do you debug an Ingress that isn't routing traffic correctly?

Source answer:
- `kubectl describe ingress <name>` — confirms the rules Kubernetes actually registered and often flags backend errors directly.
- Confirm the backing **Service** exists and has healthy **Endpoints**: `kubectl get endpoints <service>` — empty endpoints means no pods are Ready, which looks like an Ingress problem but isn't.
- Check the Ingress **controller's own pod logs** (e.g. `ingress-nginx-controller`) — it logs the actual upstream it tried to hit and the response/error.
- If TLS is involved: confirm the referenced TLS Secret exists, is in the same namespace, and the cert actually matches the host.
- Check annotations — rewrite-target, path type (`Prefix` vs `Exact` vs `ImplementationSpecific`), and any custom snippet annotations are common sources of "it looks right but doesn't match."
- Confirm external DNS actually points at the Ingress controller's LB IP/hostname — sounds obvious, but stale DNS is a frequent false alarm here.

Round-2 · net

28. kube-proxy has iptables and IPVS modes — what's the difference and why would it matter for troubleshooting?

Source answer:
- **iptables mode** (long-time default) — programs DNAT rules directly in iptables for every Service/Endpoint combination. Simple and reliable at small-to-medium scale, but rule evaluation is roughly O(n) with the number of rules, so with thousands of Services, connection setup latency can measurably increase.
- **IPVS mode** — uses the Linux kernel's IPVS (IP Virtual Server), which uses hash-table lookups instead of a long sequential rule chain, so it scales much better with a large number of Services, and supports more load-balancing algorithms (round-robin, least connection, etc.).
- Why it matters operationally: if you're diagnosing weird intermittent latency at scale that correlates with Service count rather than actual traffic, kube-proxy mode and rule-processing overhead is worth checking (`kubectl logs -n kube-system <kube-proxy-pod>` shows which mode it's running, and `iptables-save | wc -l` vs `ipvsadm -Ln` gives a sense of scale).

Round-2 · net

29. Explain what a CNI plugin does and name a few real ones.

Source answer:
CNI (Container Network Interface) is the plugin spec kubelet calls to actually set up a pod's network namespace — assign an IP, wire up interfaces/routes, and (for supporting plugins) enforce NetworkPolicy.
- **Calico** — BGP or overlay (IPIP/VXLAN) modes, strong NetworkPolicy enforcement, popular in on-prem and cloud alike.
- **Cilium** — eBPF-based, can do L3-L7 policy and even replace kube-proxy entirely for Service load-balancing; strong observability story (Hubble).
- **Flannel** — simple VXLAN overlay, historically did **not** enforce NetworkPolicy on its own (often paired with Calico for policy).
- **AWS VPC CNI** — gives pods real VPC IPs directly instead of an overlay (great performance, but can hit ENI/IP exhaustion on small subnets — a real EKS troubleshooting scenario).
- **Azure CNI** — similar real-VNet-IP model for AKS.

Round-2 · net

30. Explain NetworkPolicy and the default-allow model — what's a mistake people commonly make?

Source answer:
By default, Kubernetes networking is **fully open** — any pod can talk to any other pod in the cluster, with no NetworkPolicy required.
- A NetworkPolicy only starts restricting traffic **once it selects a pod** — the moment any policy's `podSelector` matches a pod, that pod becomes default-deny for whichever direction(s) (ingress/egress) that policy covers, and only explicitly allowed traffic gets through.
- Rules are defined by `podSelector`, `namespaceSelector`, and/or `ipBlock`, combined with ports/protocols.
- **Common mistake:** writing an ingress-only NetworkPolicy and assuming egress is now also locked down — it isn't, unless you explicitly write an egress policy too (or explicitly declare an empty egress policy to lock it down).
- Also easy to miss: NetworkPolicy enforcement depends entirely on the **CNI** supporting it — applying a NetworkPolicy on a CNI that doesn't enforce it (plain Flannel, for instance) silently does nothing.

Round-2 · net

31. Explain the basics of a service mesh and what problem it solves.

Source answer:
A service mesh (Istio, Linkerd) injects a sidecar proxy (commonly Envoy) into every pod, transparently intercepting all inbound/outbound traffic, giving you infrastructure-level capabilities without touching application code:
- **mTLS** between services automatically.
- **Retries, timeouts, circuit breaking** configured declaratively.
- **Fine-grained traffic shifting** — weighted routing for canary releases, mirroring traffic for testing.
- **Observability** — golden-signal metrics (rate/errors/duration) and distributed tracing per-hop, consistently, across every service without each team instrumenting it themselves.
- Trade-off worth mentioning in an interview: added latency per hop, extra CPU/memory overhead per pod, and genuine operational complexity — it's a deliberate cost/benefit decision, not a free upgrade.

Round-2 · net

32. Two pods in the same namespace can't talk to each other. What's your checklist?

Source answer:
- Confirm the target pod is actually **Ready** — `kubectl get endpoints <service>` should list its IP; if it's missing, the Service (and thus DNS) won't route to it even though the pod exists.
- Confirm the **Service selector** labels actually match the pod's labels exactly (a common typo/case-mismatch issue) — `kubectl get pods --show-labels` vs `kubectl describe svc`.
- Confirm the app is actually **listening on the port** you think it is, from inside the container: `kubectl exec -it <pod> -- ss -tulnp` (or `netstat`).
- Check for a **NetworkPolicy** selecting either pod that doesn't explicitly allow this traffic.
- Isolate with a raw test: `kubectl exec -it <source-pod> -- curl -v <target-service>:<port>` or spin up a temporary debug pod with `nc`/`curl` to separate "DNS problem" from "connectivity problem" from "app not listening" from "policy blocking it".

sc
Round-2 · sc

33. Users report the application is slow. Walk me through your troubleshooting approach end to end.

Source answer:
This is the classic "structured order" question — the interviewer is grading the **order and completeness** of your approach as much as the specific fixes. A strong answer moves from broad signals to narrow root cause:

- **Check Grafana dashboards first** — get the big picture: is this global or localized to one service/region/customer?
- **Review Prometheus metrics** — request rate, error rate, and p95/p99 latency (the RED method) for the affected service(s).
- **Check pod CPU/memory** — is the workload actually resource-constrained, or CPU-throttled (CFS quota) even with headroom showing on paper?
- **Verify HPA status** — is it scaling at all, is it hitting `maxReplicas`, or reacting too slowly for the traffic pattern?
- **Check node utilization** — rule out noisy-neighbor or node-level saturation separate from the pod's own limits.
- **Review ingress/load balancer errors** — 5xx rates, connection timeouts, upstream errors at the edge.
- **Check application logs** — for exceptions, slow-query warnings, thread pool exhaustion messages.
- **Check database latency** — slow queries, lock waits, connection pool exhaustion are extremely common root causes for "everything is slow" that shows low CPU.
- **Verify external/third-party API health** — a slow downstream dependency will manifest as your service being "slow" even though your own code and infra are fine.
- **Identify root cause and implement a fix** — and say explicitly whether you'd mitigate first (scale up, roll back a recent deploy) versus root-cause first, based on customer impact.

Always state the order out loud in the interview — that structure itself is what's being evaluated.

Round-2 · sc

34. Why would CPU utilization stay low while latency is high? Interviewers ask this specifically to catch people who assume 'high latency = need more CPU.'

Source answer:
High latency with low CPU means the bottleneck is somewhere the CPU graph can't show you:

- **I/O wait** — the process is blocked waiting on disk or network, not computing; CPU sits idle during that wait. Check `iostat -x` / `%iowait` at the OS level.
- **Downstream dependency latency** — a slow database query or slow external API call means your service's threads are just waiting, not burning CPU.
- **Lock contention / thread pool exhaustion** — requests queue up waiting for a limited pool (DB connections, worker threads) while the CPU that's actually running is nearly idle.
- **DNS resolution delays** — each outbound call doing a slow/uncached DNS lookup adds latency invisible to a CPU chart.
- **CPU throttling from a Kubernetes CPU limit** — this is a great callback: a container can be throttled by its CFS quota in short bursts that don't show up as sustained high CPU% on an averaged graph, but absolutely shows up as tail latency (check `container_cpu_cfs_throttled_periods_total`).
- **Garbage collection pauses** — a stop-the-world GC pause adds latency without necessarily showing as high average CPU over a wider time window.
- **Undersized concurrency** — the app simply isn't configured to run enough requests in parallel, so extra requests queue instead of using available CPU.

Round-2 · sc

35. How do you identify a memory leak in production?

Source answer:
- Graph `container_memory_working_set_bytes` (or the app's own memory gauge) over hours/days — the signature of a leak is memory that **climbs and never comes back down**, even during low-traffic windows or after garbage collection, as opposed to a normal sawtooth that resets.
- Compare the trend **across pod restarts** — if every replica independently climbs at a similar rate regardless of traffic, that's a strong leak signal versus a traffic-correlated pattern.
- Go language-specific from there:
- **Java** — heap dump (`jmap -dump`) and analyze with a tool like Eclipse MAT; look for growing collections/caches holding references.
- **Go** — `pprof` heap profile, look for goroutine leaks (a goroutine that never exits holds its stack and any captured references forever).
- **Python** — `tracemalloc` or `memory_profiler`, watch for unbounded module-level caches or circular references keeping objects alive.
- Common root causes across languages: unbounded in-memory caches with no eviction policy, unclosed connections/file handles, and event listeners/callbacks that are registered but never deregistered.

Round-2 · sc

36. What dashboards would you build for a microservice, and why those specifically?

Source answer:
Structure the answer around the **RED method** for a request-driven service (Rate, Errors, Duration) and the **USE method** for the underlying resources (Utilization, Saturation, Errors) — naming these frameworks explicitly signals seniority.

- **Request rate** (RPS), broken down by endpoint/status code.
- **Error rate** (% of 4xx/5xx, or app-defined failures) — as a percentage, not just a raw count, so it's meaningful across traffic swings.
- **Latency percentiles** — p50/p95/p99 (never just an average — averages hide the tail that customers actually feel).
- **Resource utilization/saturation** — CPU (including throttling), memory, and for Kubernetes specifically, current replica count next to HPA target, so you can see scaling behavior at a glance.
- **Dependency health** — latency and error rate for each downstream call (DB, cache, external API) so you can immediately tell if a "your service is slow" alert is actually a dependency's problem.
- **Saturation of internal resources** — connection pool usage, thread pool queue depth, cache hit ratio.
- Bonus point in an interview: mention **deploy annotations** on the dashboard (a vertical marker every time a new version ships) — it turns "latency went up at 2:14pm" into an instant correlation with "we deployed at 2:13pm."

Round-2 · sc

37. Tell me about a production outage you handled. What's a strong structure for this answer?

Source answer:
I can't hand you a real outage story since that has to come from your own experience — but here's a structure senior/AVP interviewers are explicitly listening for, so you can slot in a real incident of yours:

- **Impact/Situation** — what broke, who was affected, and how you or the team first became aware (paged, customer report, dashboard alert).
- **Immediate triage** — how you assessed severity/blast radius in the first few minutes, and who you looped in.
- **Mitigation before root cause** — what you did to stop customer impact quickly (rollback, failover, scaling, feature flag) even before you fully understood why — this ordering (mitigate first, root-cause after) is exactly what senior interviewers want to hear, since it shows you prioritize customer impact over intellectual curiosity in the moment.
- **Root cause** — what you found once you had time to dig, and how you confirmed it (not just a guess).
- **Prevention/follow-through** — the concrete action items that came out of the postmortem, and — this is the part people skip — that you (or the team) actually tracked them to completion.
- Keep it concrete: specific commands/dashboards you used, specific numbers (how long to detect, how long to mitigate), and what you'd do differently next time.

Round-2 · sc

38. How do you approach an unfamiliar production issue at 3am, on a system you didn't build?

Source answer:
- **Acknowledge the page immediately** — even before you know anything, so others know it's being handled.
- **Assess blast radius/severity first** — is this affecting all customers or a subset, is data at risk, is it revenue-impacting — this determines urgency and who else needs to be pulled in.
- **Ask "what changed recently?"** before diving deep — a recent deploy, config change, feature flag flip, or infra change correlating with the onset is by far the fastest lead to a fix, and is often faster than reading unfamiliar code at 3am.
- **Check the golden-signal dashboards** (rate/error/latency) to localize which service/component is actually the source versus just a downstream symptom.
- **Favor mitigation over full understanding** — a rollback or failover that resolves customer impact is the right move even if you don't yet know the exact root cause; you can (and should) investigate fully afterward with a clear head.
- **Communicate status** on a regular cadence even if the update is "still investigating, no customer-facing change yet" — silence is worse than a boring update.
- **Write things down as you go** — what you checked and ruled out — both to avoid re-checking the same thing under stress and to make the eventual postmortem accurate.

Round-2 · sc

39. You're seeing intermittent 5xx errors, but only under peak load. How do you debug it?

Source answer:
Intermittent-under-load is a classic saturation signature rather than a straightforward bug — look for something that's fine at normal traffic but runs out at the edges:

- **Resource saturation at peak** — CPU throttling (CFS quota) that only bites once traffic pushes utilization past the limit; check `container_cpu_cfs_throttled_periods_total`, not just raw CPU%.
- **Connection/thread pool exhaustion** — a pool sized for average load runs out under peak, and the errors you see are pool-timeout errors dressed up as generic 5xxs.
- **HPA reaction lag** — autoscaling has real latency (metric collection interval + scale-up time + new pod readiness time); a sharp traffic spike can outrun it entirely. Consider pre-scaling for known peak windows, or a predictive/KEDA-based approach for bursty traffic.
- **Downstream dependency saturation** — your service might be fine, but its database or a shared cache hits its own limit first under the same peak load.
- **Thundering herd** — e.g. a cache TTL expiring right at peak, causing a stampede of requests to the database simultaneously.
- **Rate limiting misconfiguration** — check whether your own or a dependency's rate limiter is triggering exactly at these load levels.
- Correlate the 5xx spike timestamps precisely against all of the above in Grafana rather than guessing — the overlap tells you which one it actually is.

lin

Round-2 · lin

40. How do you troubleshoot high CPU usage on a Linux host?

Source answer:
- `top` / `htop` — identify the offending process(es) quickly, sorted by CPU.
- `mpstat -P ALL 1` — per-core breakdown; a single maxed-out core with others idle points to a single-threaded bottleneck rather than genuine overall saturation.
- Break down **where** the CPU time is going: user time (app code), system time (kernel/syscalls — heavy syscall usage can mean excessive I/O or context switching), iowait (waiting on disk, not actually "using" CPU), and **steal time** (%st) — high steal on a cloud VM means the hypervisor is giving CPU cycles to other tenants, a noisy-neighbor problem you can't fix locally.
- `pidstat -p <pid> 1` for a focused per-process time series.
- `perf top` (if available) to see which functions are actually hot, when you need to go deeper than "which process."
- Fix depends on root cause: scale/right-size, fix an inefficient loop or hot code path, or — for steal time — move to a different instance type or talk to the cloud provider.

Round-2 · lin

41. Load average is high but CPU usage looks low. What's happening?

Source answer:
Load average on Linux counts processes that are **runnable or in uninterruptible sleep (D state)** — not just processes actively burning CPU. So a high load average with low CPU% almost always means a lot of processes are blocked waiting on something else, most commonly disk I/O.
- `ps aux | awk '$8 ~ /D/'` (or `ps -eo stat,pid,cmd | grep "^D"`) to find processes stuck in D state.
- `iostat -x 1` — look at `%util` and `await` on the relevant disk; near-100% util with high await confirms disk is the bottleneck.
- Could also be NFS/network-mounted storage hanging, or in some cases heavy uninterruptible waits on a misbehaving driver.
- The interview-worthy distinction to state clearly: load average measures **demand for the CPU (including things waiting to use it)**, not **CPU utilization** — they can and do diverge.

Round-2 · lin

42. How do you investigate high memory usage or an out-of-memory situation on a Linux host?

Source answer:
- `free -h` — pay attention to the **available** column specifically, not just "free"; a lot of "used" memory is often reclaimable page cache, which Linux will happily give back under pressure — `available` already accounts for that.
- `cat /proc/meminfo` for the detailed breakdown (Buffers, Cached, Slab, etc.).
- Per-process: `ps aux --sort=-%mem | head` or `smem` for a more accurate shared-memory-aware view.
- If the OOM killer actually fired: `dmesg -T | grep -i "out of memory"` or `journalctl -k | grep -i oom` — this shows exactly which process was killed and the memory state at the time.
- Distinguish **application-level** OOM (a single process's own leak/growth) from **system-level** memory pressure (many processes collectively exceeding physical memory, possibly compounded by a Kubernetes node overcommitting pods without proper limits).

Round-2 · lin

43. Disk is showing full but df -h says there's space, or vice versa — walk through disk troubleshooting.

Source answer:
- `df -h` — overall filesystem usage by mount point.
- `df -i` — **inode** usage specifically; you can have plenty of free bytes but zero free inodes (common with huge numbers of tiny files), which also presents as "no space left on device" even though `df -h` shows free capacity.
- `du -sh /* 2>/dev/null | sort -rh` (or scoped to a suspect directory) to find what's actually consuming space.
- The classic trap: a large file was **deleted** but is still held open by a running process, so the space isn't actually freed at the OS level even though it's gone from any directory listing. Find these with `lsof | grep deleted` (or `lsof +L1`) — the fix is either restarting the process (releases the handle) or, if you can't restart it, truncating the still-open file descriptor via `/proc/<pid>/fd/<fd>`.
- For genuinely full disks: log rotation gaps, forgotten core dumps, and orphaned container images/layers are the usual suspects.

Round-2 · lin

44. How do you diagnose slow disk I/O?

Source answer:
- `iostat -x 1` — the key columns are `%util` (how busy the device is) and `await` (average time per I/O request, including queueing) — high await with high util means the disk itself is the bottleneck; high await with low util can mean a slow backend (e.g. network-attached storage) rather than local contention.
- `iotop` — per-process view of who's actually generating the I/O.
- Consider the storage type: local NVMe vs network-attached block storage (EBS, Azure Disk) behave very differently under load, and network-attached storage has its own IOPS/throughput ceilings that don't show up as "the disk is broken," just as consistently elevated latency.
- For a suspected failing physical disk: `smartctl -a /dev/sdX` for SMART health data, and check `dmesg` for I/O errors.
- Cross-reference with `dstat` or Grafana's disk panels if you want a combined view rather than juggling multiple tools live during an incident.

Round-2 · lin

45. Walk me through your general network troubleshooting toolkit on Linux, in order.

Source answer:
Work up the stack from basic reachability to application-level detail:

- `ping <host>` — basic ICMP reachability (note: some environments block ICMP, so a failure here isn't definitive on its own).
- `traceroute` / `mtr <host>` — see the path and where latency/loss is actually introduced hop by hop; `mtr` is generally preferred live since it keeps sampling continuously.
- `nc -zv <host> <port>` or `telnet <host> <port>` — confirms actual **TCP** reachability on a specific port, independent of whether the application behind it responds correctly.
- `curl -v <url>` — application-layer detail: DNS resolution time, TCP connect time, TLS handshake time, TTFB, all broken out separately, which is invaluable for isolating *which* layer is slow.
- `ss -tulnp` — what's actually listening locally, and on which interface/port, from the target host's side.
- `dig` / `nslookup` — isolate DNS specifically, since DNS problems masquerade as "the service is down" constantly.
- `tcpdump -i eth0 port 443` — packet-level capture when you need to see retransmits, resets, or malformed packets directly, as a last resort when everything above looks fine but something is still wrong.

Round-2 · lin

46. An application logs 'Too many open files.' How do you troubleshoot and fix it?

Source answer:
- Check the currently configured limit: `ulimit -n` for the shell/process, or for a specific running process, `cat /proc/<pid>/limits`.
- Check actual current usage: `lsof -p <pid> | wc -l`, or `ls /proc/<pid>/fd | wc -l`.
- Two possible causes, and they need different fixes:
- **The limit is genuinely too low** for a legitimate, well-behaved workload — raise it via `/etc/security/limits.conf` (for login sessions) or the systemd unit's `LimitNOFILE=` (for services, since systemd units don't inherit `limits.conf` by default).
- **The application is leaking file descriptors** — sockets or files opened and never closed. `lsof -p <pid>` repeated over time showing a steadily growing count (especially many entries to the same remote host/port, or many identical file paths) is the signature of a leak, and the fix belongs in the application code, not the ulimit.

Round-2 · lin

47. What are zombie processes and are they actually a problem?

Source answer:
A zombie (`<defunct>` in `ps`) is a process that has already terminated, but its **parent** hasn't yet called `wait()` to read its exit status — the kernel keeps a minimal process table entry around until that happens.
- A zombie itself consumes essentially no resources (no memory, no CPU) beyond a PID table slot — a handful of zombies usually isn't an emergency.
- It **does** become a real problem if the parent never reaps children and zombies accumulate over time, potentially exhausting the system's PID limit, which then blocks new process creation entirely — that's the actual failure mode to worry about.
- Diagnosis: `ps aux | grep Z` to find them, then look at the **parent** PID (`PPID`) — the bug is in the parent's signal handling (not properly catching `SIGCHLD` / calling `wait()`), not in the zombie itself.
- Fix: fix the parent application's child-reaping logic; as an immediate mitigation, restarting/killing the parent process causes `init`/systemd to adopt and reap any of its remaining zombies.

Round-2 · lin

48. How do you use journalctl and systemctl to troubleshoot a failing service?

Source answer:
- `systemctl status <service>` — quick snapshot: is it active/failed, how long has it been in that state, and the last handful of log lines inline.
- `journalctl -u <service> -f` — follow logs live as you reproduce or wait for the issue.
- `journalctl -u <service> --since "10 min ago"` (or `--since`/`--until` with explicit timestamps) to scope to an incident window precisely instead of scrolling.
- `systemctl is-failed <service>` — useful in scripts/automation to check state programmatically.
- If a service is in a failed state and you've fixed the underlying issue: `systemctl reset-failed <service>` before `systemctl restart <service>`, otherwise some restart-rate-limiting configurations will refuse to restart it again immediately.
- `journalctl -k` specifically for kernel messages (OOM kills, hardware errors, driver issues) when the problem might be below the application/service layer entirely.

sh

Round-2 · sh

49. Write a command/script to delete log files older than 30 days.

Source answer:

```
find /var/log/myapp -type f -name "*.log" -mtime +30 -print
```

Always run with `-print` first to see exactly what would be deleted before adding the destructive action. Once verified:

```
find /var/log/myapp -type f -name "*.log" -mtime +30 -delete
```

- `-mtime +30` means "modified more than 30 days ago" (based on 24-hour periods).
- `-delete` is generally preferred over `-exec rm -f {} \;` since it's built into `find` and avoids spawning a subprocess per file, but either works.
- In production, this is usually better handled by `logrotate` with a `maxage`/rotation count rather than a standalone cron+find script — mention that as the more idiomatic long-term answer if asked.

Round-2 · sh

50. Write a command to find the top 10 largest files in a directory tree.

Source answer:

```
find /path -type f -exec du -h {} + | sort -rh | head -n 10
```

Or, simpler for smaller trees:

```
du -ah /path | sort -rh | head -n 10
```

- `du -h` gives human-readable sizes; `sort -rh` sorts by human-readable size correctly (reverse, treating "10M" vs "2G" properly) rather than sorting the strings lexically.
- If you specifically want only files (not directory subtotals), the `find ... -exec du` form is more precise since plain `du -ah` also includes directory-level rollups.

Round-2 · sh

51. Write a script that restarts a service if it stops running.

Source answer:

```
#!/bin/bash
SERVICE="nginx"
LOG="/var/log/service_watch.log"

if ! systemctl is-active --quiet "$SERVICE"; then
systemctl restart "$SERVICE"
echo "$(date '+%F %T'): $SERVICE was down, restarted" >> "$LOG"
fi
```

Run via cron every minute for a simple watchdog.

Worth saying explicitly in an interview: in a real systemd environment, the more idiomatic fix is usually the unit file itself —

```
[Service]
Restart=on-failure
RestartSec=5
```

— letting systemd handle it natively is more reliable than a polling script, since systemd reacts immediately on the actual failure event instead of waiting for the next cron tick, and it won't race with a restart that's already in progress.

Round-2 · sh

52. Write a script that alerts if CPU usage exceeds 90%.

Source answer:

```
#!/bin/bash
THRESHOLD=90
CPU_IDLE=$(top -bn1 | grep "Cpu(s)" | awk -F',' '{print $4}' | awk '{print $1}')
CPU_USAGE=$(awk "BEGIN {print 100 - $CPU_IDLE}")

if awk "BEGIN {exit !($CPU_USAGE > $THRESHOLD)}"; then
echo "High CPU usage: ${CPU_USAGE}% on $(hostname)" | mail -s "CPU Alert: $(hostname)" ops@example.com
fi
```

- Parsing `top` output for idle% and subtracting from 100 is the classic approach; `mpstat 1 1` is a more robust/parseable alternative if available.
- Using `awk` for the floating-point comparison since bash's built-in arithmetic doesn't handle decimals.
- In practice at senior level, mention that this kind of static threshold alert is better implemented as a Prometheus alerting rule (`rate`/`avg` over a window, with a `for:` duration to avoid flapping on a brief spike) rather than a cron+mail script — this shows you know when NOT to reach for a shell script.

Round-2 · sh

53. Write a script/command to read and process a CSV file.

Source answer:

```
while IFS=',' read -r name value status; do
echo "Name: $name | Value: $value | Status: $status"
done < file.csv
```

To skip a header row:

```
tail -n +2 file.csv | while IFS=',' read -r name value status; do
echo "Name: $name | Value: $value | Status: $status"
done
```

- `IFS=','` sets the field separator for `read`.
- Important caveat worth stating out loud: this naive approach breaks on CSVs with quoted fields containing embedded commas — for anything beyond a simple, well-controlled internal file, use `awk -F','` carefully or, better, a real CSV parser (Python's `csv` module) rather than hand-rolled shell splitting.

Round-2 · sh

54. Write a command to parse a log file for a specific pattern within a time window.

Source answer:

Count and rank error lines:

```
grep "ERROR" app.log | awk '{print $1, $2}' | sort | uniq -c | sort -rn
```

Extract only a specific time window (assuming a `YYYY-MM-DD HH:MM:SS` prefix, which sorts and compares correctly as a string):

```
awk '$0 >= "2026-07-21 08:00:00" && $0 <= "2026-07-21 09:00:00"' app.log | grep "ERROR"
```

- For structured/JSON logs, `jq` is the better tool than `awk`/`grep` — e.g. `jq -r 'select(.level=="error") | .message' app.log` — worth mentioning that log format changes which tool is appropriate.

Round-2 · sh

55. Write a command to count HTTP status codes from an access log.

Source answer:

```
awk '{print $9}' access.log | sort | uniq -c | sort -rn
```

- The field number depends on the actual log format in use (this assumes a standard/combined nginx log where the status code lands in field 9) — always confirm the field position against the real `log_format` directive rather than assuming, since a customized format shifts every field number.
- For a quick "how many 5xx in the last hour" style check combined with a time filter:

```
awk '$9 ~ /^5/' access.log | wc -l
```

Round-2 · sh

56. Write a script to monitor disk usage and send an email alert when it crosses a threshold.

Source answer:

```
#!/bin/bash
THRESHOLD=85

df -hP | grep -vE '^Filesystem|tmpfs|cdrom' | awk '{print $5, $6}' | while read -r usep partition; do
pct="${usep%\%}"
if [ "$pct" -ge "$THRESHOLD" ]; then
echo "Disk usage critical on $partition: ${usep}" | mail -s "Disk Alert: $(hostname)" ops@example.com
fi
done
```

- `df -hP` (`-P` for POSIX output format) avoids line-wrapping issues on filesystems with long names that break the naive `awk` field parsing.
- Stripping the `%` sign from the usage field before the numeric comparison is the detail people often forget and then wonder why the `-ge` comparison fails.
- Worth mentioning as a senior-level addition: piping the alert to a webhook (`curl -X POST` to a Slack/PagerDuty endpoint) is generally preferred over `mail` in modern setups, since it's more likely to actually reach someone in a timely, on-call-aware way.

Round-2 · sh

57. What makes a shell script 'production-safe'? What do you always include?

Source answer:

- `set -euo pipefail` at the top — exit on any command failure, treat unset variables as errors, and make a failure anywhere in a pipeline (not just the last command) cause the whole pipeline to fail.
- Quote all variable expansions (`"$var"`, not `$var`) to avoid word-splitting/globbing surprises, especially with filenames containing spaces.
- Use `mktemp` for any temporary files instead of hardcoding a path in `/tmp` (avoids collisions and symlink-attack style issues).
- Add a `trap` for cleanup on exit (`trap 'rm -f "$TMPFILE"' EXIT`) so temp resources are cleaned up even if the script errors out partway through.
- Log meaningfully — timestamps, what action is being taken, and clear success/failure at the end — so a 3am on-call engineer can tell what the script actually did without re-reading the source.
- Design for **idempotency** — running it twice in a row (e.g. because cron overlapped, or a retry fired) should be safe, not double-apply an action.
- Use explicit exit codes and check them (`$?` or direct `if command; then`) rather than assuming success.
- Avoid parsing `ls` output for scripting logic — use `find` or globs instead, since `ls` output isn't designed to be machine-parsed reliably (breaks on filenames with spaces/newlines).

prom

Round-2 · prom

58. Explain Prometheus's architecture end to end.

Source answer:
- The **Prometheus server** itself does three things: scrapes targets on an interval, stores samples in its local **TSDB**, and exposes a **PromQL** query engine (used by its own UI, Grafana, and Alertmanager rule evaluation).
- **Service discovery** figures out *what* to scrape — the Kubernetes SD mechanism, file-based SD, cloud-provider SD, etc. — instead of hardcoding a static target list.
- Each target exposes metrics on an HTTP `/metrics` endpoint in Prometheus's text exposition format.
- **Alertmanager** is a separate component — Prometheus evaluates alerting rules and pushes firing alerts to Alertmanager, which handles dedup/grouping/routing/silencing independently.
- **Pushgateway** is the one exception to the pull model — for short-lived batch jobs that would be gone before a scrape could ever reach them, they push their result to the Pushgateway, and Prometheus scrapes *that* instead.
- For long-term storage/global queries beyond a single Prometheus's local retention, `remote_write` streams samples out to a system like Thanos, Cortex, Mimir, or VictoriaMetrics.
- **Grafana** sits on top as the visualization layer, querying Prometheus's HTTP API — Grafana itself stores no metrics data.

Round-2 · prom

59. Why is Prometheus pull-based instead of push-based, and what's the Pushgateway for?

Source answer:
Pull (scrape) advantages:
- Prometheus itself controls scrape frequency/load centrally, instead of every app independently deciding how often to push.
- A failed scrape is itself a meaningful signal — "target down" is directly observable, whereas with push you can't easily distinguish "nothing to report" from "the sender is dead."
- Service discovery driven target management scales cleanly — Prometheus finds new targets automatically as they appear (e.g. new pods).

Where pull breaks down: short-lived **batch jobs** that finish and exit before Prometheus's next scheduled scrape could ever reach them. The **Pushgateway** solves this specifically — the job pushes its final metric values to the Pushgateway once, and Prometheus scrapes the Pushgateway on its normal interval instead of the job directly. It's explicitly *not* meant as a general push-metrics ingestion point for long-running services.

Round-2 · prom

60. What are exporters? Name several you've likely worked with.

Source answer:
Exporters translate a system's or application's native metrics into Prometheus's exposition format, since not everything speaks Prometheus natively.
- **node_exporter** — host/OS-level metrics (CPU, memory, disk, network) for any Linux machine.
- **kube-state-metrics** — Kubernetes **object state** (deployment replica counts, pod phase, etc.) — distinct from resource usage.
- **cAdvisor** — container resource usage metrics, built directly into kubelet, exposed per-node.
- **blackbox_exporter** — active probing (HTTP/TCP/ICMP/DNS) from the outside, useful for synthetic uptime/latency checks rather than instrumented-from-inside metrics.
- **mysqld_exporter / redis_exporter** — database/cache-specific metrics (connections, replication lag, hit ratio, etc.).
- The distinction worth stating clearly: **cAdvisor tells you resource usage; kube-state-metrics tells you object/desired-state** — they're commonly confused but answer different questions.

Round-2 · prom

61. Explain what Alertmanager does that Prometheus's own alerting rules don't.

Source answer:
Prometheus's rule evaluation only decides **when an alert should fire** (an expression true for a sustained `for:` duration) — everything about what happens *after* that is Alertmanager's job:
- **Deduplication** — the same alert firing from multiple Prometheus replicas (in an HA setup) doesn't become multiple pages.
- **Grouping** — batching related alerts (e.g. by cluster/service label) into a single notification instead of paging once per affected pod.
- **Inhibition** — suppressing lower-severity alerts that are made redundant by a higher-severity one already firing (e.g. don't page for "high latency" separately if "service down" for the same service is already firing).
- **Silencing** — a temporary, time-boxed mute, typically used during planned maintenance.
- **Routing** — a tree of matchers (by label) that decides which receiver (Slack, PagerDuty, email, a generic webhook) gets which alert, and with what wait/repeat intervals.

Round-2 · prom

62. Recording rules vs alerting rules — what's the difference?

Source answer:

- **Recording rules** precompute a PromQL expression on a schedule and save the result as a brand-new time series. Used for expressions that are expensive to compute repeatedly (heavy aggregations over high-cardinality data) or reused across many dashboards/alerts — computing once and storing it is far cheaper than every dashboard panel recalculating the same aggregation live.
- **Alerting rules** evaluate an expression and, when it's true continuously for the configured `for:` duration, fire an alert that gets sent to Alertmanager.
- They're often layered together: a recording rule computes something like a service's error rate, and an alerting rule then simply checks that pre-computed series against a threshold — cleaner and cheaper than embedding the full aggregation inside the alert expression itself.

Round-2 · prom

63. Explain relabeling in Prometheus and what it's used for.

Source answer:

Relabeling rewrites, filters, or generates labels **before** a target is scraped (`relabel_configs`) or **before** a scraped sample is stored (`metric_relabel_configs`).

- Common uses: dropping targets/metrics you don't actually want to keep (cost/cardinality control), rewriting a label's value into something more useful, and — very common in Kubernetes SD — promoting Kubernetes metadata labels (which arrive as `__meta_kubernetes_*` labels, prefixed with `__` so they're normally internal/discarded) into real, queryable labels like `namespace` or `pod`.
- The double-underscore prefix convention is worth knowing cold: any label starting with `__` is dropped after relabeling unless you explicitly keep/rename it — this trips people up constantly when Kubernetes SD "isn't showing the labels I expected."

Round-2 · prom

64. Explain federation, remote_write, and HA pairs — when would you use each?

Source answer:

- **Federation** — one Prometheus scrapes a curated, aggregated subset of metrics from another Prometheus's `/federate` endpoint. Useful for a hierarchical "global view" rolled up from many smaller Prometheus instances, but it doesn't give you full-resolution long-term data — considered somewhat legacy now that remote-write-based solutions exist.
- **remote_write** — continuously streams every sample to an external long-term-storage backend (Thanos, Cortex, Mimir, VictoriaMetrics) for durability, much longer retention, and global querying across many Prometheus instances/clusters.
- **HA pairs** — run two (or more) identical Prometheus replicas scraping the exact same targets independently, so a single Prometheus crashing doesn't cause a gap in data or missed alerts. You then need a query layer (like Thanos Querier) that can deduplicate the two nearly-identical series when you query across both.
- One-line summary if asked to pick: federation for a hierarchical rollup view, remote_write for durable long-term/global storage, HA pairs purely for availability.

Round-2 · prom

65. Walk through these PromQL examples and explain what each one does: rate(http_requests_total[5m]), sum(rate(container_cpu_usage_seconds_total[5m])) by (pod), histogram_quantile(0.95, ...).

Source answer:

```
rate(http_requests_total[5m])
```

`http_requests_total` is a counter (always increasing). `rate()` computes the **per-second average rate of increase** over the trailing 5-minute window — this is the standard way to turn a raw counter into something graphable/alertable, since a raw counter's absolute value isn't meaningful on its own.

```
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)
```

Same `rate()` idea applied to per-container CPU seconds, then `sum by (pod)` aggregates multiple containers within a pod into one series per pod — giving you total CPU usage per pod rather than per container.

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

This computes the **p95 latency** from a Prometheus histogram metric — `_bucket` series carry a cumulative count per `le` (less-than-or-equal) bucket boundary, and `histogram_quantile` interpolates the 95th percentile from those bucket counts. Note it needs `by (le)` preserved in the aggregation — dropping that label breaks the calculation.

Worth mentioning if asked: `rate()` vs `irate()` — `irate()` uses only the **last two** data points, making it more reactive/spiky (good for live graphing of fast-changing rates), while `rate()` smooths over the whole window (better for alerting, where you don't want to fire on a single noisy sample).

dk

Round-2 · dk

66. Explain multi-stage Docker builds and why they matter.

Source answer:

A multi-stage build uses multiple `FROM` statements in one Dockerfile — an early stage has the full toolchain (compilers, build dependencies, dev packages) needed to build the artifact, and a later, separate stage copies only the finished build output into a minimal runtime base image.

```
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .

FROM gcr.io/distroless/base-debian12
COPY --from=builder /app/myapp /myapp
ENTRYPOINT ["/myapp"]
```

- The final image contains none of the compiler/build tooling — dramatically smaller image size and a much smaller attack surface (fewer packages = fewer CVEs to track).
- Also ties directly into image optimization generally: pick a minimal base (alpine/distroless) for the final stage, and combine this with a `.dockerignore` and minimizing the number of layers in the final stage.

Round-2 · dk

67. Explain Docker image layers and caching, and how you'd structure a Dockerfile for fast builds.

Source answer:
Every instruction in a Dockerfile creates a new, cached **layer**. On a rebuild, Docker reuses a cached layer as long as the instruction and its build context haven't changed — the moment one layer changes, every layer **after** it must be rebuilt too, even if unrelated.

Practical implication for ordering: put the **least frequently changing** instructions first.
```
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
```
Copying just the dependency manifest and installing dependencies **before** copying the full source means the (usually slow) install step stays cached across builds where only application code changed — reversing this order (copying everything first) busts the dependency-install cache on every single code change, making every build slow.

Round-2 · dk

68. ENTRYPOINT vs CMD — what's the difference and how do they interact?

Source answer:
- **ENTRYPOINT** defines the fixed executable that always runs when the container starts.
- **CMD** provides default arguments — which *can* be overridden by anything passed at `docker run` time.
- Used together, they give you a sensible default that's still flexible:
```
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8080"]
```
Running the image normally launches `python app.py --port 8080`; running `docker run myimage --port 9090` overrides just the args, keeping the fixed entrypoint intact.
- If you only set **CMD** with no ENTRYPOINT, any arguments given to `docker run` replace the entire CMD, not just append to it — a common point of confusion.

Round-2 · dk

69. Explain Docker volumes vs bind mounts vs tmpfs, and how this maps to Kubernetes.

Source answer:
- **Named volumes** — managed by Docker itself, persist independently of any container's lifecycle, portable and easy to back up. Roughly maps to a Kubernetes **PersistentVolume/PersistentVolumeClaim**.
- **Bind mounts** — a specific host path mapped directly into the container. Great for local development (live code reload), but ties you to that host's filesystem layout — the rough Kubernetes analog is `hostPath`, which is generally discouraged in production since it breaks the "pods are portable across nodes" assumption.
- **tmpfs** — in-memory, fully ephemeral, gone when the container stops — useful for sensitive temporary data you explicitly don't want touching disk. Kubernetes has an equivalent `emptyDir` with `medium: Memory`.

Round-2 · dk

70. Explain Docker networking modes, and how Kubernetes networking differs fundamentally.

Source answer:
- **bridge** (default) — containers get an internal IP on a virtual bridge, NAT'd to the host; you explicitly map ports (`-p 8080:80`) to reach them from outside the host.
- **host** — the container shares the host's network namespace directly, no isolation, no port mapping needed, lowest latency, but no port-conflict protection either.
- **none** — no networking at all.
- **Kubernetes is fundamentally different**: every **pod** gets its own real, routable IP in a flat cluster network (no per-host NAT the way default Docker bridge networking works) — this is what the CNI plugin sets up, and it's why "which Docker network mode is this pod using" isn't really the right question once you're on Kubernetes; the model has moved to "every pod is its own fully addressable network endpoint."

Round-2 · dk

71. How do Docker/container resource limits actually get enforced, and how does that connect to the 'low CPU, high latency' problem?

Source answer:
- Memory and CPU limits (`--memory`, `--cpus`, or in Kubernetes, `resources.limits`) are enforced via Linux **cgroups**.
- Exceeding the **memory** limit → the kernel's OOM killer terminates the container (this is the OOMKilled mechanism covered elsewhere).
- The **CPU** limit is enforced via the CFS (Completely Fair Scheduler) **quota/period** mechanism — the container gets a fixed slice of CPU time per period (e.g. 100ms every 100ms for a 1.0 CPU limit) and, if it tries to use more, it's **throttled** — paused until the next period — rather than killed.
- This throttling is exactly the mechanism behind the "CPU usage looks low/fine on an averaged graph, but latency spikes" scenario: throttling happens in short bursts that don't show up clearly on a coarse-grained average CPU% chart, but absolutely show up as added tail latency. The metric to check specifically is `container_cpu_cfs_throttled_periods_total` (or the throttled-time equivalent), not just raw CPU%.

ci

Round-2 · ci

72. Explain the full pipeline: GitHub → Jenkins/GitHub Actions → Docker → Registry → Argo CD → Kubernetes.

Source answer:
- A push/PR merge to GitHub triggers **CI** (Jenkins or GitHub Actions) — build, run tests/lint, and on success, build a **Docker image** tagged meaningfully (commit SHA is the most traceable choice, sometimes combined with semver).
- The image is pushed to a **registry** (ECR/ACR/Harbor/Docker Hub).
- CI then updates a **separate GitOps repository's** manifests/Helm values with the new image tag (either directly, or automatically via something like Argo CD's Image Updater) — this is the crucial handoff point between CI and CD.
- **Argo CD**, running in the cluster and continuously watching that GitOps repo, detects the drift between declared desired state (in Git) and actual cluster state, and syncs the cluster to match.
- The core principle to name explicitly: this is **GitOps** — Git is the single source of truth for desired state, and a pull-based operator (not the CI pipeline pushing directly `kubectl apply`) reconciles the cluster to match it. This gives you a full audit trail via Git history, and rollback is as simple as `git revert`.

Round-2 · ci

73. What are your rollback options when a deployment goes bad?

Source answer:
- **Immediate/tactical**: `kubectl rollout undo deployment/<name>` — uses the ReplicaSet revision history Kubernetes already kept; fastest option, but it's a manual, out-of-band action that Git/GitOps doesn't know happened.
- **GitOps-native (preferred for anything managed by Argo CD/Flux)**: revert the offending commit in the GitOps repo and let the operator auto-sync back — keeps Git as the actual source of truth and avoids drift between what Git says and what's running.
- **Helm**: `helm rollback <release> <revision>` if the app is Helm-managed.
- Operational hygiene point worth mentioning: set `revisionHistoryLimit` deliberately (not 0, and not unbounded) so you always have recent history to roll back to without keeping every revision forever.

Round-2 · ci

74. Explain blue-green deployment — pros, cons, when you'd choose it.

Source answer:
Two full, identical environments exist simultaneously — **Blue** (currently live) and **Green** (the new version) — fully provisioned and warmed up before any real traffic hits Green.
- Cutover is instant: flip a Service selector or DNS/load-balancer target from Blue to Green all at once.
- Rollback is equally instant — flip back to Blue, which is still fully running and untouched.
- Cost: you're running **2x the infrastructure** during the transition window, which can be significant for large services.
- Best fit: major or higher-risk releases where you want a guaranteed instant, complete rollback path and can tolerate the doubled resource cost temporarily.

Round-2 · ci

75. Explain canary deployment — pros, cons, when you'd choose it over blue-green.

Source answer:

Gradually shift a small percentage of real traffic to the new version — e.g. 5% → 25% → 50% → 100% — monitoring error rate and latency at each step before proceeding.

- Blast radius is naturally limited — if the new version is broken, only a small slice of traffic/users ever see it, versus blue-green's all-or-nothing cutover.
- Can be automated end-to-end (Argo Rollouts, Flagger) to analyze Prometheus metrics at each traffic-weight step and automatically pause or roll back if error rate/latency degrades beyond a threshold, without a human watching every step.
- More operationally complex to set up than blue-green — needs a mechanism capable of weighted traffic splitting (a service mesh, or an Ingress controller that supports it), not just a simple selector flip.
- Choose canary when you want gradual, metrics-gated risk reduction on frequent releases; choose blue-green when you want a single clean instant cutover/rollback, often for less frequent, higher-stakes releases.

Round-2 · ci

76. How do you handle secrets in a CI/CD pipeline securely?

Source answer:

Never commit plaintext secrets to Git — even in a private repo, since Git history is effectively forever. Real options:

- **Sealed Secrets** — encrypt a Secret client-side with a public key; the encrypted form is safe to commit to Git, and only a controller running in-cluster (holding the private key) can decrypt it back into a real Kubernetes Secret.
- **External Secrets Operator** — pulls secrets at runtime from a real secrets store (Vault, AWS Secrets Manager, Azure Key Vault) and materializes them as native Kubernetes Secrets, so Git never holds any secret material at all, encrypted or not.
- **SOPS** — encrypts specific values within a YAML/JSON file (backed by KMS/PGP), so a values file can be committed with individual fields encrypted.
- **CI-native secret stores** (GitHub Actions secrets, Jenkins credentials store) injected as environment variables at build/deploy time — fine for the CI stage itself, but not a substitute for a real secrets management story once things land in the cluster.
- Whatever the mechanism, also make sure secrets never end up printed in CI logs (a very common accidental leak) or baked into an image layer.

Round-2 · ci

77. A CI/CD pipeline is failing intermittently — how do you troubleshoot flaky pipelines?

Source answer:

- Check for **shared/non-idempotent test state** — tests that pass or fail depending on execution order or leftover state from a previous run.
- Check for **resource contention on CI runners** — a runner sized too small, or too many parallel jobs contending for the same shared resource (a shared test DB, for example).
- Check for **network flakiness to external dependencies** inside tests — anything hitting a real external API/service during tests is a reliability risk; mock it.
- Check for **race conditions in parallel test execution** — tests that assume exclusive access to a resource (a fixed port, a fixed test-data row) will intermittently collide when run in parallel.
- Check **timeout values** — a step that's simply too close to its timeout under normal variance will fail "randomly" under slightly higher load.
- Add retries as a stopgap if genuinely needed, but treat that as masking the symptom — the actual fix is removing the non-determinism, and retries should be a deliberate, temporary decision, not the default answer.

aws

Round-2 · aws

78. Explain AWS VPC, subnets, and route tables at a level relevant to troubleshooting.

Source answer:

- A **VPC** is an isolated virtual network within a region.
- **Subnets** are IP ranges carved out of the VPC, each tied to a single Availability Zone.
- A subnet is "public" or "private" purely based on its **route table** — a public subnet's route table sends `0.0.0.0/0` traffic to an **Internet Gateway**; a private subnet routes `0.0.0.0/0` to a **NAT Gateway** instead (or has no default route at all, fully isolated).
- Troubleshooting angle: "why can't this instance reach the internet" is very often simply "check its subnet's route table," before looking at security groups, NACLs, or the application at all.

Round-2 · aws

79. NAT Gateway vs Internet Gateway — what's the actual functional difference?

Source answer:
- **Internet Gateway (IGW)** — allows both **inbound and outbound** internet traffic for resources that have a public IP, attached at the VPC level. This is what a public subnet routes through.
- **NAT Gateway** — allows **outbound-only** internet access for private-subnet resources that have no public IP of their own; it translates their traffic to the NAT Gateway's own public IP. Unsolicited inbound connections from the internet cannot initiate through a NAT Gateway — only responses to connections the private resource itself opened.
- One-line distinction to give in an interview: IGW is bidirectional and for public-IP resources; NAT Gateway is outbound-only and specifically exists to give private resources internet access without exposing them to inbound internet traffic.

Round-2 · aws

80. Security Groups vs NACLs — how are they different, and which do you check first?

Source answer:
- **Security Groups** — **stateful** (return traffic is automatically allowed, no need for an explicit outbound rule matching an allowed inbound one), attached at the instance/ENI level, support **allow-only** rules (no explicit deny).
- **NACLs** — **stateless** (you must explicitly allow both directions independently), applied at the **subnet** level, support both allow **and explicit deny** rules, evaluated in rule-number order (lowest number wins first match).
- In practice, Security Groups are the first line of defense and where most day-to-day access rules live; NACLs are a coarser subnet-level backstop, often left mostly open/default and only tightened for specific compliance or defense-in-depth requirements. When troubleshooting connectivity, check Security Groups first — they're the more common culprit — then NACLs if SGs look correct but traffic still isn't getting through.

Round-2 · aws

81. ALB vs NLB — when do you pick each?

Source answer:
- **ALB (Application Load Balancer)** — Layer 7, understands HTTP/HTTPS/WebSocket, supports host- and path-based routing, integrates directly with AWS WAF, and can do things like redirect rules and fixed responses natively.
- **NLB (Network Load Balancer)** — Layer 4 (TCP/UDP), built for extreme performance — millions of requests per second, ultra-low latency, a **static IP per Availability Zone**, and it preserves the client's source IP by default (ALB requires reading it from the `X-Forwarded-For` header instead).
- Pick NLB specifically for non-HTTP protocols, when you need a static IP (some enterprise firewall allow-listing requirements demand this), or for genuinely extreme throughput/latency requirements; pick ALB for essentially all normal HTTP(S) application traffic where L7 routing/WAF integration is valuable.

Round-2 · aws

82. Explain Auto Scaling Groups and how you'd troubleshoot one not scaling as expected.

Source answer:
- An **Auto Scaling Group (ASG)** maintains a desired number of EC2 instances based on a launch template, scaling via policies — **target tracking** (e.g. keep average CPU at 60%), **step scaling** (specific adjustments per threshold breached), or **scheduled** (known traffic patterns).
- Health checks (EC2 or attached ELB health checks) determine when an instance is considered unhealthy and gets replaced automatically.
- If it's "not scaling as expected": check the actual CloudWatch metric the policy is tracking (is it genuinely crossing the threshold, or is a stale/wrong metric being watched), check the ASG's min/max/desired bounds aren't already capping it, check the **cooldown period** isn't suppressing a scaling action that just fired recently, and check launch template/AMI issues aren't causing new instances to fail health checks and get cycled repeatedly instead of actually joining service.

Round-2 · aws

83. You're running EKS. What are the EKS-specific things you'd know for a troubleshooting-heavy interview?

Source answer:
- The **control plane is fully managed** by AWS (API server, etcd run across multiple AZs) — you don't operate it directly, which also means some traditional control-plane troubleshooting (etcd health, for instance) isn't something you have direct access to; you lean on AWS-provided control plane logs (enable them via CloudWatch) instead.
- Worker nodes run via **managed node groups** (EC2-based, you still patch/manage the underlying AMI lifecycle to a degree) or **Fargate** (fully serverless pods, no node management at all, mapped to specific namespaces/labels via Fargate profiles).
- **AWS VPC CNI** gives pods real VPC IP addresses directly (not an overlay) — the real troubleshooting scenario this creates: small subnets can run out of available IPs faster than expected, since every pod consumes a real VPC IP, and this shows up as pods stuck Pending with an IP-allocation-related event rather than a typical resource-scheduling failure.
- **IRSA (IAM Roles for Service Accounts)** — lets you grant fine-grained AWS permissions to a specific Kubernetes ServiceAccount (and therefore specific pods) instead of giving every pod on a node the same broad node IAM role — a security best practice worth mentioning proactively.

az

Round-2 · az

84. Explain Azure VNets and subnets, and VNet peering.

Source answer:
- A **VNet** is Azure's equivalent of an AWS VPC — an isolated virtual network within a region, subdivided into **subnets**.
- **VNet peering** connects two VNets so resources in each can communicate using private IPs as if on the same network, without traffic traversing the public internet — commonly used to connect a hub (shared services) VNet to multiple spoke VNets, or to connect VNets across regions.
- Peering is **not transitive** by default — if VNet A peers with B, and B peers with C, A cannot reach C through B automatically; that needs its own explicit peering or a routing/hub solution.

Round-2 · az

85. Explain NSGs (Network Security Groups) and how they compare to what you'd use in AWS.

Source answer:
- An **NSG** is Azure's primary network-filtering construct — roughly analogous to a combination of AWS Security Groups and NACLs.
- **Stateful** (like AWS Security Groups) — return traffic for an allowed connection is automatically permitted.
- Can be applied at the **subnet level or the NIC level** (or both — and when both apply, the more restrictive combined result wins), which is more flexible than AWS's instance-only Security Group model.
- Rules are evaluated by **priority number**, lowest number processed first, first match wins — and Azure has some default rules already in place (e.g. deny-all-inbound-from-internet by default) that you can't delete, only override with a higher-priority rule.

Round-2 · az

86. Explain Private Endpoints vs Service Endpoints in Azure.

Source answer:
- **Private Endpoint** — provisions an actual **private IP address inside your VNet** for a PaaS service (Storage Account, Azure SQL, etc.) via Azure Private Link — traffic never touches the public internet at all, and the service can even be made unreachable via its public endpoint entirely.
- **Service Endpoint** — keeps traffic on the Azure backbone network (rather than the public internet) and lets you restrict a PaaS resource's firewall to only your VNet/subnet — but the resource **still has a public endpoint/DNS name**, just with network-level access restricted.
- The practical difference to state clearly: Private Endpoint gives the service a private IP in your network (stronger isolation); Service Endpoint is a lighter-weight, VNet-restricted access rule on a resource that still has a public identity.

Round-2 · az

87. Azure Load Balancer vs Application Gateway — how does this map to what you know from AWS?

Source answer:
- **Azure Load Balancer** — Layer 4 (TCP/UDP), high-performance, low-latency — directly analogous to an AWS NLB.
- **Application Gateway** — Layer 7, HTTP(S)-aware, host/path-based routing, integrates with Azure WAF — directly analogous to an AWS ALB.
- Same fundamental decision as AWS: pick Application Gateway for standard HTTP(S) application traffic needing L7 routing/WAF, pick Azure Load Balancer for non-HTTP protocols or when you need raw L4 performance.

Round-2 · az

88. Explain Azure Monitor and its components.

Source answer:
Azure Monitor is the umbrella platform — roughly Azure's answer to AWS CloudWatch, but broader in scope:
- **Metrics** — near-real-time numeric time series for resource performance.
- **Log Analytics** — a workspace where logs from many sources land, queried with **KQL (Kusto Query Language)** — this is where you'd go for anything beyond a simple metric, like correlating events across services.
- **Application Insights** — APM/tracing specifically for applications — request rates, dependency call latency, exceptions, distributed tracing — the closest analog to something like an APM agent plus distributed tracing combined.
- **Alerts** — rules on metrics or log queries that trigger **Action Groups** (email, SMS, webhook, Logic App, ITSM integration) when conditions are met.

Round-2 · az

89. You mentioned Azure Synapse experience — what would an SRE actually be responsible for operationally?

Source answer:

Azure Synapse combines data warehousing (**dedicated SQL pools** and **serverless SQL pools**) with big-data processing (**Spark pools**) and orchestration (**pipelines**, similar in concept to Data Factory) in one workspace. From an SRE/operations angle rather than a data-engineering one:

- **Monitoring pipeline runs** — failures, retries, and duration trends, since a pipeline silently taking 3x longer is often the first sign of an upstream data or capacity problem.
- **Managing dedicated SQL pool scaling (DWUs)** — dedicated pools are billed whether queried or not, so **pausing** idle dedicated pools (and resuming on a schedule or on-demand) is a real, recurring cost-management responsibility, not just a performance one.
- **Handling pipeline failures/retries** — building alerting around failed pipeline runs, and understanding retry/backoff behavior so transient upstream issues don't require manual intervention every time.
- **Capacity/concurrency troubleshooting** — Synapse has real concurrency limits per pool; queueing/throttling under heavy concurrent use is a legitimate "why is this suddenly slow" investigation, similar in spirit to a database connection pool problem.

Round-2 · az

90. Explain VMSS and Key Vault, and how they typically work together in an SRE context.

Source answer:

- **VMSS (Virtual Machine Scale Sets)** — Azure's equivalent of an AWS Auto Scaling Group: a group of identical VM instances that scale in/out based on metrics or a schedule, behind a load balancer.
- **Key Vault** — centralizes secrets, keys, and certificates, with fine-grained access policies/RBAC controlling who and what can read them.
- How they typically connect in practice: a VM (or an AKS pod) uses a **Managed Identity** to authenticate to Key Vault and pull secrets/certs at runtime — meaning no credentials are ever baked into an image, a startup script, or a config file. In AKS specifically, this is done via **Workload Identity** (the modern approach, replacing the older AAD Pod Identity), binding a Kubernetes ServiceAccount to an Azure AD identity so specific pods can access Key Vault without broad node-level credentials.

db

Round-2 · db

91. A database suddenly becomes slow. Walk through your triage.

Source answer:

- Check for **active/blocking queries**: `SHOW PROCESSLIST` (MySQL) or `pg_stat_activity` (Postgres) to see what's actually running right now and for how long.
- Check for **lock waits/blocking sessions** specifically — a single long-running transaction holding a lock can stall many others behind it even though the DB itself "looks" fine on CPU.
- Check the **slow query log** for anything newly appearing that wasn't there before.
- Check **connection count vs max_connections** — approaching the limit causes queueing/rejection that looks like generic slowness from the app's side.
- Ask "what changed recently" — a new deploy that introduced an unindexed query, or a schema migration that dropped/altered an index, is one of the most common root causes for a sudden (rather than gradual) slowdown.
- Check **host-level resources** — CPU, and specifically disk I/O and available memory (page cache eviction/swapping under memory pressure hits every query at once, which matches a "suddenly everything is slow" pattern well).
- If reads are served from a **replica**, check replication lag separately — the "database" being slow might actually be a lagging replica serving stale/slow reads while the primary is fine.

Round-2 · db

92. Explain connection pool exhaustion — symptoms, causes, and fixes.

Source answer:

- **Symptoms**: "too many connections" errors from the database, or the application appearing to hang while actually just waiting for a connection to free up from its own pool.
- **Causes**: connections not being released properly (a leak — missing `close()`/not using a context manager or try-finally), a pool sized too small for actual concurrent demand, long-running queries holding a connection far longer than typical, or a genuine traffic spike exceeding what the pool/DB was sized for.
- **Fixes**: fix the leak first if one exists (this is the most common actual root cause, not undersizing), tune pool size against the database's real `max_connections` (with headroom for other clients), add statement timeouts so a stuck query can't hold a connection indefinitely, and consider an external pooler like **PgBouncer** (Postgres) to multiplex many application connections over fewer actual database connections.

Round-2 · db

93. Explain database deadlocks — how do they happen and how do you troubleshoot them?

Source answer:

A deadlock is two (or more) transactions each holding a lock the other one needs, so neither can proceed — the database engine detects this cycle and forcibly kills one transaction (the "victim") with a deadlock error to break the cycle.
- Diagnose via the engine's own deadlock reporting: Postgres's `log_lock_waits`/deadlock log entries, or MySQL's `SHOW ENGINE INNODB STATUS` (which includes the last deadlock's details directly).
- Fixes:
- Enforce **consistent lock ordering** across all transactions that touch the same tables/rows (if every transaction always locks table A before table B, a cycle can't form).
- Keep transactions as **short** as possible — the longer a transaction holds locks, the more likely it collides with another.
- Review the **isolation level** in use — a higher isolation level increases lock scope/duration and can increase deadlock likelihood.
- Add **application-level retry logic** specifically for deadlock-victim errors, since a retried transaction very often succeeds cleanly the second time.

Round-2 · db

94. How do you investigate a specific slow/long-running query?

Source answer:

- Run `EXPLAIN ANALYZE` (Postgres) or `EXPLAIN` (MySQL) on the actual query to see the real execution plan — not just the theoretical one.
- Look specifically for a **sequential/full table scan** on a large table where an index scan would be expected — this is the single most common finding.
- Check whether table **statistics are stale** (`ANALYZE` in Postgres) — the query planner makes bad decisions off outdated row-count/distribution estimates, which can silently make a previously-fast query slow after enough data changes.
- Check for **lock contention** specifically affecting this query rather than a generic resource issue.
- Look at the query from the **application** side too — a single "slow query" reported by the app is sometimes actually an **N+1 query pattern** (many small fast queries executed in a loop) rather than one genuinely expensive query, and the fix there is at the application/ORM level, not the database.

Round-2 · db

95. Explain replication lag — causes and mitigation.

Source answer:

Replication lag is the delay between a write landing on the **primary** and that same write becoming visible on a **replica**.
- Common causes: the replica genuinely can't keep up with write volume (historically a real constraint with single-threaded replication apply in MySQL), the replica is under-provisioned relative to the primary, a long-running query on the replica is blocking replication apply, or heavy write bursts simply outrun steady-state replication throughput.
- Monitor directly: `Seconds_Behind_Master` (MySQL) or the lag exposed via `pg_stat_replication` (Postgres).
- Mitigation: reduce/batch heavy write load where possible, right-size replica hardware relative to the primary, enable parallel replication apply where the engine supports it, and — importantly for application design — avoid routing latency-sensitive reads that need the absolute latest data to a replica that might be lagging; route those to the primary instead.

Round-2 · db

96. A database host's disk is filling up unexpectedly. What do you check?

Source answer:

- **WAL/binlog accumulation** — if replication is broken or a replica has fallen far behind, the primary can't safely purge old write-ahead logs/binary logs, and they'll accumulate indefinitely until disk fills — this is one of the most common "disk suddenly full on a DB host" root causes, and it's really a replication problem wearing a disk-space costume.
- **Runaway temp files** — large sorts/joins/hash operations that spill to disk can generate very large temporary files under unexpected query patterns.
- **Table/index bloat** (Postgres specifically) — if autovacuum isn't keeping up, dead tuples accumulate and inflate on-disk size well beyond the "logical" data size.
- Immediate mitigation vs long-term fix: freeing space right now (old log/temp file cleanup, or resizing the disk) buys time, but the actual fix is addressing whichever root cause above is generating the growth, or it will simply fill up again.

inc

Round-2 · inc

97. Walk through the full incident lifecycle.

Source answer:
- **Detection** — an alert fires, or a customer/internal report surfaces the problem.
- **Triage/Acknowledge** — someone picks it up, does an initial severity assessment, and lets others know it's being handled.
- **Investigation/Diagnosis** — narrowing down what's actually wrong.
- **Mitigation** — stopping the customer-facing bleeding (rollback, failover, scale-up, feature flag) — deliberately **before** full root cause is required, if impact is ongoing.
- **Resolution** — the actual, complete fix (which may happen after mitigation, not necessarily at the same time).
- **Postmortem/RCA** — a blameless write-up of what happened and why.
- **Follow-up action items** — concrete, owned, due-dated tasks coming out of the postmortem.
- **Closure** — confirming action items are actually done, not just written down.

Round-2 · inc

98. How do severity levels typically work, and what determines the severity of an incident?

Source answer:
Exact naming varies by org, but the general shape is consistent — something like Sev1 (critical) down to Sev3/Sev4 (minor):
- **Sev1** — full outage or major customer-facing impact, data integrity at risk, or significant revenue/SLA impact — immediate all-hands response, paged regardless of time of day.
- **Sev2** — significant but partial impact — a subset of customers/functionality affected.
- **Sev3/Sev4** — minor, no real customer impact, can typically be handled during business hours on a normal ticket/backlog basis.
- The determining factors to name explicitly: **scope of customer impact**, **data integrity/security risk**, and **revenue/SLA exposure** — not simply "how technically interesting or severe the underlying bug is."

Round-2 · inc

99. How do you handle stakeholder communication during an active incident?

Source answer:
- Communicate on a **regular, predictable cadence** (e.g. every 30 minutes) even when the update is just "still investigating" — silence during an incident is worse than a boring status update, since stakeholders will assume the worst in a vacuum.
- Structure each update clearly: **what we know**, **what we're doing about it**, and **when the next update will be**.
- A dedicated **Incident Commander** role — someone who owns communication and coordination specifically — lets the engineers actually fixing the problem stay heads-down instead of context-switching to answer "any updates?" every few minutes.
- Avoid speculation in customer-facing updates — state confirmed facts and current status, not unconfirmed theories about root cause, which can (and does) get quoted back later if wrong.

Round-2 · inc

100. Explain Root Cause Analysis (RCA) and how you'd run one well.

Source answer:
- The goal is to go **past the immediate trigger** to the systemic/contributing factors — a technique like the "5 Whys" is useful for pushing past the first, surface-level answer.
- Distinguish three separate things clearly in the writeup: the **trigger** (what specifically kicked it off, e.g. a config change), the **root cause** (the underlying condition that made that trigger capable of causing an outage at all), and **contributing factors** (things that worsened impact or slowed detection/response, even if they didn't directly cause the failure).
- It must be **blameless** — the goal is understanding the system and process gaps that allowed the incident, not identifying a person to blame — a blame-oriented RCA process actively discourages the honest, detailed input needed to actually find the real cause.

Round-2 · inc

101. What belongs in a good postmortem document?

Source answer:
- A clear **timeline** — when detection happened, when mitigation started, when it was actually resolved, with real timestamps.
- **Impact** — quantified where possible (duration, % of requests affected, customers affected, revenue impact if known).
- **Root cause** — the actual underlying reason, distinguished from the immediate trigger.
- **What went well / what didn't** — honestly, including gaps in monitoring/alerting that delayed detection, not just the technical failure itself.
- **Action items** — each with a specific **owner** and a **due date** — vague action items ("improve monitoring") without ownership are the single most common reason postmortems don't actually prevent a repeat.
- Written in a **blameless** tone throughout, and shared broadly enough that the organizational learning actually spreads beyond the immediate team involved.

Round-2 · inc

102. Postmortems generate action items — how do you make sure they actually get done?

Source answer:
- Track them in the **same system** the team already uses for real work (the actual sprint board/ticket tracker), not a separate postmortem document that nobody revisits.
- Assign a clear **owner and due date** at the time the postmortem is written — not "the team will look into this."
- Periodically **review open action items** from past postmortems in a recurring forum (even briefly) so they don't silently age out and get forgotten.
- Prioritize the categories that most directly prevent a repeat or speed up the next detection/response: closing a **monitoring/alerting gap** that delayed detection, adding an **automated safeguard** (circuit breaker, rate limit, automated canary analysis/rollback) so the failure mode is caught by a machine next time rather than a human, and updating **runbooks** with what was actually learned.
- Some orgs validate this loop explicitly with **game days/chaos engineering** — deliberately re-triggering a similar failure mode in a controlled way to confirm the fix (and the runbook, and the alert) actually works, rather than assuming it does.

SearchStax Senior SRE Interview Guide

Scenario

Scenario · Scenario

103. Describe a real production incident where your first understanding of the problem turned out to be incorrect — initial symptoms, why you suspected the wrong component, what changed your direction, the actual root cause, and the follow-up improvements.

Source answer:
Situation: Prometheus fired a High Response Time alert on a customer-facing API roughly 15 minutes after a routine release. CPU on the pods had jumped from ~40% to ~90%, and pods were showing elevated restart counts.

Initial assumption: Everything pointed at the new release: CPU spike immediately after deploy, latency climbing in lockstep, no infrastructure-level alerts firing. I assumed a memory leak or an inefficient code path shipped in the release.

What changed my thinking: I went through kubectl top pod, kubectl describe pod, kubectl logs, and kubectl get events — no OOM kills, no exceptions, no crash loops. The application itself was healthy. That ruled out the code-regression theory, so I widened the search to dependencies and checked Prometheus metrics for the database tier, which showed connection-pool saturation.

Root cause: A schema migration bundled with the release had silently dropped an index. A query that normally ran in ~20ms was now taking several seconds, backing up the connection pool, which back-pressured the app and looked like a CPU/app problem from the outside.

Fix: Recreated the missing index. Latency and CPU normalized within minutes — no application rollback needed.

Prevention: Added slow-query alerting, made index changes part of migration review/checklist, and added a synthetic post-deploy check that specifically watches DB query latency for the first 30 minutes after any release — not just app-level metrics.

Lesson: Correlation isn't causation. 'It happened right after the deploy' pointed at the app, but the actual signal — connection pool saturation — was one layer deeper. Now I always validate app, infra, and dependency metrics as three separate hypotheses before committing to a root cause.

Scenario · Scenario

104. You join as the first SRE. No structured monitoring, no meaningful dashboards, inconsistent alerting, frequent complaints about instability. Walk through your first 30 days.

Source answer:

Days 1–7 — Listen before instrumenting: I wouldn't touch tooling in week one. I'd interview on-call engineers and support/product about where the pain actually is — which service gets escalated most, what a 'bad day' looks like, what people currently do manually to check health. In parallel, I'd inventory what exists: any metrics being collected but unused, existing alert rules and their noise/false-positive rate, and recent incident history if any postmortems exist. Goal: a prioritized list of the top 3-5 reliability risks, backed by what people actually experience, not by what looks technically interesting to me.

Days 8–14 — Instrument the golden signals, cut alert noise: For the top 1-2 services identified, stand up the four golden signals (latency, traffic, errors, saturation) in Prometheus/Grafana — even a rough dashboard beats none. At the same time, I'd audit existing alerts and kill or tune anything that's fired more than it's mattered; a team that's been ignoring pages for months has already trained itself not to trust alerts, and that has to be fixed before adding more signals.

Days 15–21 — Define SLIs/SLOs where it matters most: Rather than boiling the ocean, I'd pick the single most customer-visible service and define one or two SLIs (e.g. request success rate, p95 latency) with a realistic SLO based on historical behavior, not an aspirational number. This gives leadership a measurable, defensible way to talk about reliability instead of 'it feels flaky.'

Days 22–30 — Runbook culture and visible progress: Write the first 2-3 runbooks for the most common incident types identified in week 1, and start a lightweight weekly reliability review — what fired, what didn't, what changed. I'd close the 30 days with a short writeup to leadership: risks found, what's now measured, what's fixed, and a prioritized backlog for the next quarter.

Balancing immediate vs. long-term: Immediate firefighting (killing noisy pages, fixing the most complained-about issue) buys credibility and calm fast; the SLO/runbook work is what compounds. I'd timebox firefighting deliberately so it doesn't eat the whole 30 days — the goal isn't to be a hero, it's to leave behind a system that's more observable and more predictable than the one I found.

Lesson: The fastest way to lose trust as the first SRE is to show up with a big monitoring rollout before anyone believes you understand their pain. Visibility work has to follow listening, not precede it.

Scenario · Scenario

105. Describe the most technically challenging Linux/infra/production issue you've personally investigated — what made it hard, what misled you, what worked, what didn't, and what you learned.

Source answer:

Situation: A service began throwing intermittent upstream-timeout errors under load — not constant, maybe 2-3% of requests, and only during peak traffic windows. CPU, memory, and pod counts all looked completely normal, which made it genuinely confusing.

What was hard about it: Nothing in the standard dashboards pointed anywhere. No resource exhaustion, no error spikes at the LB, no obvious pattern by pod or node. My first instinct — that it was a downstream service being slow — was wrong; the downstream service's own latency metrics were clean.

What misled me initially: Because the errors were timeouts, I spent time chasing network policy and service mesh timeout configs, assuming a misconfigured retry/timeout budget. That ruled itself out once I found the timeouts weren't correlated with any specific route or service pair.

What narrowed it down: I used tcpdump and strace on an affected pod during a live occurrence and noticed the delay was happening before the outbound connection was even established — pointing at DNS resolution rather than the network call itself. Checking kube-dns/CoreDNS metrics showed a spike in query latency and a rising count of dropped/retried queries that exactly tracked the peak traffic windows.

Root cause: Under peak load, the number of concurrent DNS lookups from application pods exceeded CoreDNS's configured capacity (and conntrack table pressure on the nodes was compounding it), causing intermittent resolution delays that surfaced as upstream timeouts several layers up the stack.

Fix: Scaled CoreDNS replicas, enabled NodeLocal DNSCache to cut cross-node DNS traffic, and tuned conntrack limits on the affected node pool.

Lesson: The layer where a symptom appears (application timeout) is often several hops away from the layer where the cause lives (DNS/conntrack). When the obvious metrics are all clean, that's a signal to go one level lower in the stack — packet capture, syscall trace — rather than re-checking the same dashboards.

Scenario · Scenario

106. Describe a time you strongly disagreed with a technical/architectural/operational decision — why, how you raised it, the tradeoffs, the outcome, and what it taught you.

Source answer:
Situation: Ahead of a major release, the team wanted to skip the planned resilience/failover testing to protect the ship date — the reasoning was that the changes were 'low risk' and testing could happen after launch if needed.

Why I disagreed: The release touched a component that fed into failover logic, which is exactly the kind of change where 'looks low risk' and 'is low risk' diverge. Skipping the one test designed to catch failover regressions felt like removing the safety net specifically for the release most likely to need it.

How I raised it: I didn't just say 'no' — I brought a scoped alternative: instead of the full test suite, a targeted chaos experiment (using Gremlin) against just the failover path, time-boxed to a few hours, run in a lower environment first. I framed it in terms the team already cared about: cost of a production failover failure vs. cost of a few hours of testing time.

Tradeoffs / outcome: Leadership agreed to the scoped test rather than the full suite — a compromise on scope, not on doing it at all. The test caught a real gap: a timeout value that hadn't been updated to match the new component's startup time, which would have caused failover to fail during exactly the kind of event it exists to handle. We fixed it before release.

Lesson: Disagreement lands better as a scoped counter-proposal than as a blocker. 'Here's a smaller version of what I'm asking for, and here's the specific risk it addresses' gets a yes far more often than 'we can't ship without full testing.' It also taught me that resilience testing isn't a checkbox — it's most valuable exactly when a team feels pressure to skip it.

Scenario · Scenario

107. A critical outage is happening, but dashboards/logging/monitoring are also unavailable. Walk through the first 15-20 minutes.

Source answer:
First 60 seconds — declare and orient: I'd declare the incident immediately rather than waiting to understand it — that starts the clock on communication and gets the right people looking. I'd post a short, honest status: 'Investigating a customer-impacting outage; observability tooling is also degraded, using fallback methods, updates every 10-15 min.' Setting that expectation early prevents a flood of 'any update?' pings.

Minutes 1–5 — cheapest signals first: Before touching anything internal, I'd check the cloud provider's status page and any third-party dependency status pages — a chunk of 'our monitoring is down' incidents are actually a shared upstream provider issue, and this is the fastest way to rule that in or out.

Minutes 5–10 — validate the edge: With dashboards unreliable, I'd go straight to the load balancer / ingress layer via CLI or cloud console — target health checks, 5xx rate at the LB if it's still reporting, and current connection counts. That tells me whether the problem is 'nothing is reaching the backend' vs. 'backend is reachable but failing,' which splits the investigation in two very different directions.

Minutes 10–15 — raw signals over dashboards: I'd fall back to direct log tailing (SSH/kubectl logs/journalctl on affected hosts or pods) rather than the centralized logging UI, and basic OS-level tools — top, vmstat, ss, df -h — on a representative host to rule out resource exhaustion. Even without Grafana, the raw signals are still there; only the aggregation layer is down.

Minutes 15–20 — narrow blast radius, start the war room: By this point I'd have a rough hypothesis (edge vs. backend vs. dependency vs. resource exhaustion) and would pull in whoever owns that area, spin up a dedicated incident channel/bridge, and assign one person to own customer/stakeholder comms so the responders can stay heads-down on the fix. I'd also explicitly note in the channel which normal signals are unavailable, so nobody wastes time trying to pull up a dashboard that isn't there.

Lesson: Losing observability tooling during an outage is really two incidents at once. Treat it that way: the missing tooling is itself worth flagging for a fast-follow fix, but in the moment, fall back to the layer below the dashboard — the CLI, the raw logs, the provider status page — because the actual signals rarely all disappear at once, only the convenient view of them does.

Linux

Technical · Linux

108. A process is consuming high CPU but you don't know which thread — how do you find out?

Source answer:
top -H -p <pid> to see per-thread CPU, then convert the thread ID to hex and cross-reference with /proc/<pid>/task/<tid>/stack or use `perf top -p <pid>` to see which function is hot.

Technical · Linux

109. How does the OOM killer decide which process to kill?

Source answer:
It computes an oom_score per process (based on memory usage, adjusted by oom_score_adj) and kills the highest scorer. You can protect critical processes by setting oom_score_adj lower, or move them to a separate cgroup with a memory limit so they fail predictably instead of taking the node down.

Technical · Linux

110. Disk usage shows 100% full but `du` doesn't add up — what's going on?

Source answer:
Classic deleted-but-open-file-handle issue: a process still has a file descriptor open on a deleted file, so the space isn't released. Find it with `lsof +L1` or `lsof | grep deleted`, then restart or signal the owning process to release the handle.

Technical · Linux

111. Explain the difference between load average and CPU utilization.

Source answer:
CPU utilization is percent of CPU time used; load average is the number of processes runnable or in uninterruptible sleep (e.g. waiting on disk I/O) averaged over 1/5/15 minutes. High load with low CPU usually points to I/O wait, not compute pressure.

Technical · Linux

112. How do you debug a service that won't start under systemd?

Source answer:
`systemctl status <service>` for the immediate error, `journalctl -u <service> -xe` for full logs, and if it's silent, run the unit's ExecStart command manually in the foreground to see stdout/stderr directly.

Technical · Linux

113. What causes TIME_WAIT socket buildup and why does it matter?

Source answer:
Happens on the side that initiates connection close, holding the socket for 2xMSL to handle stray packets. Under high connection churn it can exhaust ephemeral ports. Mitigate with connection pooling/keep-alive, tuning net.ipv4.tcp_tw_reuse, or widening the ephemeral port range.

Technical · Linux

114. How do you check what's holding a file or port open?

Source answer:
`lsof -i :<port>` for ports, `lsof <path>` for files, `fuser -v <path>` as an alternative.

Technical · Linux

115. Walk through diagnosing a sudden spike in disk I/O wait.

Source answer:
`iostat -x 1` to see per-device %util and await, `pidstat -d 1` to find the offending process, then `iotop` for a live ranked view. Cross-check against recent deploys/cron jobs/log rotation that might explain the timing.

AWS

Technical · AWS

116. How would you design multi-AZ high availability for a stateless API?

Source answer:
ALB across at least 2-3 AZs, target group with health checks, ASG spanning the same AZs, and no single-AZ dependency in the data path (e.g. RDS Multi-AZ or a managed cache cluster spanning AZs).

Technical · AWS

117. EC2 instance is unreachable and you've lost the SSH key — how do you get in?

Source answer:
SSM Session Manager if the instance has the agent and an IAM role with the right policy — no key, no open port 22 needed. If SSM isn't set up, detach the root EBS volume, attach it to a rescue instance to fix/inject a new key, then reattach.

Technical · AWS

118. How do you access S3 privately from a subnet with no internet route?

Source answer:
VPC Gateway Endpoint for S3 (no NAT cost, route table entry only) or an Interface Endpoint if you need it reachable from on-prem/other VPCs too; lock it down further with an S3 bucket policy conditioned on aws:sourceVpce.

Technical · AWS

119. What's the tradeoff between an ALB and an NLB?

Source answer:
ALB operates at L7 — path/host routing, WAF integration, but adds latency and doesn't preserve client IP by default. NLB operates at L4 — ultra-low latency, preserves source IP, static IP support, but no content-based routing.

Technical · AWS

120. How do you detect and prevent Terraform state drift in AWS?

Source answer:
`terraform plan` on a schedule (CI job) to catch drift, remote state with locking (S3 + DynamoDB) to prevent concurrent writes, and IAM policies that block manual console changes to Terraform-managed resources where practical.

Technical · AWS

121. Explain IAM role vs. IAM user for a workload running on EC2/EKS.

Source answer:
Roles are assumed temporarily via STS with short-lived credentials — no long-lived secrets to leak, easy least-privilege scoping per workload (IRSA on EKS). Users have long-lived credentials and are for humans/external systems, not workloads.

Technical · AWS

122. How would you right-size an over-provisioned Auto Scaling Group?

Source answer:
Pull CloudWatch CPU/memory/network utilization over a representative window (include peak, not just average), use Compute Optimizer recommendations as a starting point, then adjust scaling policies to target utilization rather than a flat instance count.

Technical · AWS

123. Cost spiked overnight with no traffic change — how do you investigate?

Source answer:
Cost Explorer filtered by service and day-over-day, cross-reference with CloudTrail for what changed (new resource type, region, or a misconfigured autoscaling policy that over-scaled), and check for orphaned resources like unattached EBS volumes or idle NAT gateways.

Automation (Python / Bash)

Technical · Automation (Python / Bash)

124. What's your approach to deciding when a manual runbook step should become a script?

Source answer:
Rule of thumb: second time I do the same manual step during an incident, I write it down as a runbook step; second time I follow that exact runbook step, I automate it. Automating too early wastes time on something that might change; automating too late costs incident minutes repeatedly.

Technical · Automation (Python / Bash)

125. How do you make an automation script safe to run against production?

Source answer:
Dry-run mode by default, explicit --confirm/--yes flag for destructive actions, idempotency (safe to re-run), and logging every action taken so there's an audit trail if something needs to be reversed.

Technical · Automation (Python / Bash)

126. How would you write a health-check script that avoids false positives from transient blips?

Source answer:
Check N times with a short interval and require M consecutive failures before alerting (e.g. 3 of 3 or 2 of 3), rather than alerting on a single failed check — mirrors how Kubernetes/ALB health checks avoid flapping.

Technical · Automation (Python / Bash)

127. Describe a Python automation script you've built for on-call/reliability work.

Source answer:
Example pattern: a script polling the Prometheus API for a given alert's series, correlating the timestamp against recent deploys pulled from the CI system's API, and posting a summary to the incident channel — cuts the first 5 minutes of 'was this us?' triage.

Technical · Automation (Python / Bash)

128. How do you handle retries and backoff in automation that calls external APIs (AWS SDK, etc.)?

Source answer:
Exponential backoff with jitter, a capped max-retry count, and distinguishing retryable errors (throttling, 5xx) from non-retryable ones (4xx/auth) so you're not retrying something that will never succeed.

Technical · Automation (Python / Bash)

129. What's your pattern for a Bash script that needs to be safe under `set -e` with pipelines?

Source answer:
`set -euo pipefail` at the top, explicit exit-code checks after external commands, and trap on EXIT/ERR to clean up temp files or release locks even if the script dies mid-way.

CI/CD

Technical · CI/CD

130. How do you design a deployment pipeline to minimize blast radius of a bad release?

Source answer:
Canary or rolling deploy with automated health-check gates between stages, feature flags to decouple deploy from release, and an automatic rollback trigger tied to error-rate/latency SLO burn rather than requiring a human to notice first.

Technical · CI/CD

131. What belongs in a pipeline as a hard gate vs. a soft warning?

Source answer:
Hard gate: failing tests, failed security scan on critical severity, failed health check post-deploy. Soft warning: code coverage dip, non-critical lint issues — things a human should see but that shouldn't block an urgent fix.

Technical · CI/CD

132. How do you handle secrets in a CI/CD pipeline?

Source answer:
Never in the repo or plaintext env vars in the pipeline config — pull from a secrets manager (AWS Secrets Manager/Vault) at runtime with short-lived credentials scoped to that pipeline's IAM role.

Technical · CI/CD

133. Describe rolling back a bad deployment under pressure.

Source answer:
Prefer a tested one-command rollback (previous image tag / previous Helm release) over manual patching — manual fixes under pressure are how a bad deploy becomes a bad rollback. Automated rollback tied to SLO burn-rate alerts is the senior-level answer.

Technical · CI/CD

134. What's the difference between blue-green and canary deployment, and when would you pick one over the other?

Source answer:
Blue-green: full traffic switch between two complete environments, fast rollback (switch back), but doubles resource cost and doesn't catch issues that only show under partial traffic. Canary: gradual traffic shift with monitoring at each step, catches issues earlier with smaller blast radius, but takes longer to fully roll out.

Technical · CI/CD

135. How do you validate a pipeline change itself, before it affects production deploys?

Source answer:
Test pipeline changes against a non-production pipeline/branch first, and keep pipeline config in version control with review just like application code — a broken pipeline blocking all deploys during an incident is its own outage.

Observability

Technical · Observability

136. Explain the four golden signals and why they're the starting point for any dashboard.

Source answer:
Latency, traffic, errors, saturation. They're the minimum set that answers 'is the service healthy' without needing deep domain knowledge of the specific service — a good starting dashboard for any new service before deeper instrumentation exists.

Technical · Observability

137. How do you cut alert noise without missing real incidents?

Source answer:
Alert on symptoms (user-facing SLO burn) rather than every underlying cause; require sustained breach (e.g. 5 minutes over threshold, not one data point); route low-urgency signals to a dashboard/ticket instead of paging; review paged-but-not-actionable alerts weekly and tune or delete them.

Technical · Observability

138. What's a burn-rate alert and why is it better than a static threshold alert?

Source answer:
It alerts based on how fast you're consuming your error budget relative to your SLO window — e.g. a fast burn rate pages immediately even if the raw error count is still 'low,' because at that rate you'll blow the monthly budget in hours. Static thresholds miss slow, sustained degradation and over-alert on brief spikes.

Technical · Observability

139. How would you define an SLI for a search API?

Source answer:
Something like the percentage of search requests returning within a target latency (e.g. p95 < 300ms) and successfully (non-5xx) — measured from the client-facing edge, not an internal component, so it reflects actual user experience.

Technical · Observability

140. Prometheus scrape target is intermittently missing data — how do you debug it?

Source answer:
Check the target's /metrics endpoint directly for latency/errors, check Prometheus's own scrape duration/timeout settings against the target's response time, and check for cardinality explosion or the target being OOM-killed right as it's scraped.

Technical · Observability

141. What's the difference between monitoring and observability?

Source answer:
Monitoring answers known questions with pre-defined dashboards/alerts ('is CPU high'). Observability lets you answer questions you didn't anticipate, by having rich enough telemetry (metrics + logs + traces, correlated) to explore an unknown failure mode after the fact.

Technical · Observability

142. How do you avoid dashboard sprawl where nobody trusts any dashboard?

Source answer:
One canonical service-health dashboard per service, owned and reviewed regularly, with golden signals only at the top level — deeper diagnostic dashboards linked from there rather than everything crammed onto one screen.

Technical · Observability

143. How would you instrument a new service from zero for observability?

Source answer:
Start with the four golden signals via existing framework middleware/exporters, add structured logging with a correlation/request ID, then layer in tracing once there are enough downstream calls that a single trace materially helps debugging.

Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)
Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

144. How do you diagnose a sudden spike in database connection usage?

Source answer:
Check for a connection leak (app not releasing connections, e.g. after a slow query timeout) vs. a genuine traffic spike vs. a stuck long-running query holding connections — `SHOW PROCESSLIST` / `pg_stat_activity` to see what's actually held open and for how long.

Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

145. A previously fast query suddenly got slow — what's your process?

Source answer:
Check if the query plan changed (stale statistics, a dropped or unused index — as in the Q1 scenario), check table growth/bloat, and check for lock contention from a concurrent long-running transaction.

Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

146. How do you approach capacity planning for a database tier?

Source answer:
Track growth trend on the metrics that matter (connections, IOPS, storage, query latency percentiles) over weeks/months, project forward against expected traffic growth, and set an alert threshold well before the resource actually becomes a bottleneck rather than reacting at 100%.

Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

147. Explain read replicas vs. sharding, and when each is the right tool.

Source answer:
Read replicas scale read throughput and provide failover targets but don't help write throughput or dataset size. Sharding splits data across multiple primaries to scale writes/storage but adds real complexity (cross-shard queries, rebalancing) — only reach for it once read replicas and vertical scaling are genuinely insufficient.

Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

148. How do you safely run a schema migration on a live production database?

Source answer:
Backward-compatible, additive-first changes (add column, backfill, then switch reads, then drop old column) rather than a single breaking change; run during low-traffic windows for anything with lock implications; always confirm indexes survive the migration — directly the failure mode from the Q1 scenario.

Technical · Database (pick your strongest — MySQL / Postgres / MongoDB / Redis)

149. Redis is your cache layer and it goes down — what's your failure mode and how do you design for it?

Source answer:
Design the application to degrade gracefully to the source-of-truth database on cache miss/unavailability rather than hard-failing; use a circuit breaker so a Redis outage doesn't cascade into overwhelming the DB with the full uncached load.

Apache Solr basics (gap area — worth 20 minutes before the interview)

Technical · Apache Solr basics (gap area — worth 20 minutes before the interview)

150. What is a Solr collection, and how does it relate to shards and replicas?

Source answer:
A collection is the logical index (like a table). It's split into shards for horizontal scale, and each shard has replicas for redundancy and read throughput — conceptually parallel to how you'd think about partitions and replicas in any distributed data system.

Technical · Apache Solr basics (gap area — worth 20 minutes before the interview)

151. What role does ZooKeeper play in SolrCloud, and what happens if it's degraded?

Source answer:
ZooKeeper holds cluster state, shard/replica assignments, and leader election for SolrCloud. If ZooKeeper is degraded, Solr can't do leader election or config updates reliably — it's a critical coordination dependency, similar in spirit to etcd for Kubernetes.

Technical · Apache Solr basics (gap area — worth 20 minutes before the interview)

152. What are common reliability concerns specific to running Solr at scale?

Source answer:
JVM heap sizing and GC pause tuning (Solr is JVM-based and large heaps can cause long GC pauses that look like latency spikes), index/segment merge overhead affecting write throughput, and query latency degrading as index size grows without adequate sharding.

Technical · Apache Solr basics (gap area — worth 20 minutes before the interview)

153. How would you monitor Solr health?

Source answer:
JVM metrics (heap usage, GC pause time) alongside Solr-specific metrics (query latency, indexing rate, replica sync status) — exactly the golden-signals pattern applied to a JVM-based search engine instead of a typical web service.

Behavioral

Behavioral · Behavioral

154. Tell me about a time you were self-aware enough to recognize you were wrong mid-incident.

Source answer:
Point back to Q1's pattern: state the initial hypothesis, the moment the data contradicted it, and that you changed direction without ego attachment to being right. The self-awareness signal is in how fast you let go of a wrong theory, not in never having one.

Behavioral · Behavioral

155. Describe how you handle disagreement with a teammate during a live incident.

Source answer:
In the moment, defer to whoever has the clearest current evidence and keep moving — incidents aren't the place to win an argument. Raise process disagreements afterward, in the postmortem, where they can be resolved calmly and turned into a documented decision.

Behavioral · Behavioral

156. How do you perform under sustained on-call pressure, especially on a shift with limited overlap with the rest of the team (this role is 6:30 PM–3:30 AM IST)?

Source answer:
Be concrete: you default to declaring incidents early and over-communicating status rather than trying to resolve silently and report later, specifically because a remote/off-hours setup means nobody can look over your shoulder. Mention keeping a written incident log in real time so a handoff or escalation doesn't lose context.

Behavioral · Behavioral

157. Tell me about a time you had to learn an unfamiliar technology quickly under pressure.

Source answer:
Use a real example from your stack growth (e.g. picking up a new observability tool or cloud service on a deadline) — structure: what you needed to accomplish, how you scoped learning to just what was needed for the task, and what you verified before trusting it in production.

Behavioral · Behavioral

158. How do you decide what to prioritize when everything feels urgent?

Source answer:
Anchor to customer/business impact and blast radius first, then reversibility — an urgent but easily-reversible issue can wait behind an urgent, hard-to-reverse one. Say you make this explicit rather than going by gut feel, especially when coordinating with people in a different timezone who can't easily weigh in live.

Behavioral · Behavioral

159. Describe a time you had to explain a technical incident to a non-technical stakeholder.

Source answer:
Lead with customer impact and current status, not root cause mechanics; save the technical detail for engineers. Structure: what broke (plain language), what we're doing, when we'll know more — repeated on a cadence, not just once.

Behavioral · Behavioral

160. What does 'ownership' mean to you in an SRE role, concretely?

Source answer:
Point to the JD's own language — identifying parts of the system that don't scale, documenting remediation, seeing it through to long-term resolution, not just filing a ticket. Give a real example where you carried something from 'noticed a problem' to 'shipped the fix' without being asked.

Behavioral · Behavioral

161. How do you share knowledge with adjacent dev/ops teams (explicitly called out in the JD)?

Source answer:
Concrete mechanisms: runbooks that non-SREs can follow, a short postmortem summary that goes beyond just the SRE team, or pairing with a developer during an incident so they learn the debugging path, not just the fix.

SRE Interview Recall Handbook

Build & CI

5-star · Build & CI

162. What does mvn clean install do?

Source answer:

clean
→
validate
→
compile
→
test
→
package
→
verify
→
install
→
deploy
clean
Deletes the previous
target/
directory.
compile
Compiles Java source code.
test
Executes unit tests.
package
Creates a JAR or WAR.
install
Copies the artifact into the local Maven repo (
~/.m2/repository
).
package
Creates the JAR/WAR →
target/app.war
. Local only.
install
Creates the WAR AND stores it in
~/.m2/repository
so other projects can use it.
mvn clean
mvn compile
mvn test
mvn package
mvn clean package
mvn clean install
mvn deploy
Q: What's mvn dependency:tree for?
Prints the full dependency graph — used to debug version conflicts and transitive dependencies.
Q: What happens if a unit test fails during mvn install?
The build fails at the test phase and stops — package/install never run, unless -DskipTests is set (which should never be used in a real CI pipeline).

CI/CD

5-star · CI/CD

163. Walk me through your CI/CD pipeline.

Source answer:

Marked as answered correctly but unstructured — deliver it as a clean pipeline narrative, not a list.

Dev commit
→
GitHub
→
Webhook
→
Jenkins
→
Checkout
→
mvn clean install
→
Unit tests
→
SonarQube
→
Artifact build
→
Trivy scan
→
Push artifact
→
Deploy
→
Smoke test
→
Production

Q: What's the difference between Continuous Delivery and Continuous Deployment?

Delivery: every change is pipeline-verified and release-ready but a human approves the production push. Deployment: it goes to production automatically with no manual gate.

Q: Declarative vs Scripted Jenkins pipeline?

Declarative uses a fixed structured syntax (pipeline { stages { } }) — easier to read/maintain. Scripted is raw Groovy — more flexible but harder to standardize across teams.

Q: How do you handle secrets in a Jenkins/GitHub Actions pipeline?

Never hardcode. Use Jenkins Credentials Store / GitHub Actions Secrets, injected as env vars at runtime, scoped per environment, rotated regularly, and never echoed to logs.

5-star · CI/CD

164. What's the difference between SonarQube and Trivy?

Source answer:

SonarQube — static code analysis
Bugs
Code smells
Security vulnerabilities
Duplicated code
Coverage
Input:
source code
Trivy — scanner
Scans:
Docker images, Kubernetes YAML, filesystem, git repo
CVEs
OS vulnerabilities
Library vulnerabilities
Misconfiguration
Secrets

Q: Where in the pipeline does each run?

SonarQube runs right after unit tests, against source code. Trivy runs after the image is built, against the built artifact/image — shifting security left but at the right stage for each tool.

5-star · CI/CD

165. Difference between Quality Profile and Quality Gate?

Source answer:
Quality Profile
Collection of coding rules.
No duplicated code
No SQL injection
Method complexity < 15
Quality Gate
Pass / fail threshold.
Coverage > 80%
Critical issues = 0
Duplicated code < 3%
Security rating = A
If the Quality Gate fails → the pipeline fails.

4-star · CI/CD

166. How would you deploy a new version with zero downtime, and what's your rollback plan?

Source answer:
Rolling
Old pods replaced gradually by new ones. Default in Kubernetes Deployments. Low resource overhead, rollback = revert rollout.
Blue-Green
Two full environments; switch traffic (via LB/Ingress) from Blue to Green instantly. Instant rollback by switching back, but 2x infra cost.
Canary
Small % of traffic (e.g. 5%) routed to new version first; expand gradually while watching error rate/latency. Safest for risky changes.
Follow-up they'll ask: "What metrics would you watch during a canary?" → error rate, P95/P99 latency, 5xx rate, and business KPIs — tie back to Golden Signals.

Incident Mgmt

5-star · Incident Mgmt

167. Define MTTA, MTTD, MTTR, and MTBF.

Source answer:
MTTD
Mean Time To Detect — alert fires vs. when the issue actually started.
MTTA
Mean Time To Acknowledge — alert fires to someone picking it up.
MTTR
Mean Time To Resolve/Recover — issue start to full resolution.
MTBF
Mean Time Between Failures — reliability over time; higher is better.
Tie to your resume metric here: "reduced MTTD by 35% through better alerting" is exactly this term — say it explicitly.

5-star · Incident Mgmt

168. Walk me through how you run a postmortem after a major incident.

Source answer:
Timeline reconstruction
→
Impact & scope
→
Root cause (5 Whys)
→
Contributing factors
→
Action items w/ owners
→
Follow-up tracking
Say "blameless" explicitly — focus on systems/process gaps, not individuals. This phrase is a strong signal interviewers listen for.
Q: What's the "5 Whys" technique?
Ask "why" repeatedly (~5 times) on a failure to move past the symptom to the actual root cause, rather than stopping at the first plausible explanation.

4-star · Incident Mgmt

169. How do you classify incident severity, and how does escalation work?

Source answer:

SEV1
Full outage / critical business impact — all-hands, immediate page.
SEV2
Major degradation, partial impact — urgent but contained.
SEV3
Minor issue, workaround exists — normal priority.
SEV4
Cosmetic / no user impact — backlog.
Escalation: primary on-call → secondary on-call (if no ack in N minutes) → engineering manager → incident commander for SEV1.
Tools: PagerDuty/Opsgenie escalation policies.

Networking

5-star · Networking

170. Explain 500 vs 502 vs 503 vs 504.

Source answer:

2xx / 3xx
200 OK
201 Created
204 No Content
301 / 302 Redirect
4xx — Client error
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
405 Method Not Allowed
429 Too Many Requests
500
Application crashed — e.g. NullPointerException, unexpected error.
502
Bad Gateway — Nginx received an invalid response; backend not responding correctly.
503
Service Unavailable — maintenance, pods unavailable, backend overloaded.
504
Gateway Timeout — reverse proxy waited, backend never replied.
Q: 401 vs 403?
401 = not authenticated (no/invalid credentials). 403 = authenticated but not authorized for this resource.

4-star · Networking

171. Walk me through what happens when you type a URL and hit enter.

Source answer:
Browser DNS cache
→
OS resolver
→
Recursive DNS server
→
Root → TLD → Auth NS
→
IP returned
→
TCP 3-way handshake
→
TLS handshake
→
HTTP request/response
3-way handshake
SYN → SYN-ACK → ACK, then data flows.
debugging tools
dig / nslookup — DNS resolution
curl -v — full request/response trace
telnet host port — raw TCP connectivity test
traceroute / mtr — path/hop latency

4-star · Networking

172. What's the difference between a Layer 4 and Layer 7 load balancer?

Source answer:
L4 (Transport)
Routes based on IP/port (TCP/UDP). Fast, protocol-unaware. Example: Network Load Balancer (AWS NLB).
L7 (Application)
Routes based on HTTP content — headers, path, host. Enables path-based routing, SSL termination. Example: ALB, Nginx Ingress.
In Kubernetes: a
Service (LoadBalancer/NodePort)
is roughly L4; an
Ingress
is L7.

Troubleshooting

5-star · Troubleshooting

173. Walk me through troubleshooting a high CPU alert on a production pod.

Source answer:

Interviewer explicitly expected this structured walkthrough — lead with the flow, not commands first.

Alert fires
→
Confirm CPU spike
→
kubectl top
→
Recent deploy?
→
App logs
→
Node CPU?
→
Container CPU?
→
top / ps -ef
→
Java process?
→
Thread dump
→
GC logs
→
Traffic increased?
→
Memory leak?
→
Scale
→
Rollback if needed
→
RCA

kubectl top pod
kubectl top node
top
htop
ps -ef
jstack <pid>
jcmd <pid> ...
jmap -heap <pid>

5-star · Troubleshooting

174. A pod is in CrashLoopBackOff / OOMKilled / ImagePullBackOff / Pending — what do you check?

Source answer:

CrashLoopBackOff

App is crashing repeatedly right after start. Check: kubectl logs --previous, exit code, misconfigured env vars/secrets, failing readiness dependency (e.g. DB not reachable).

OOMKilled

Container hit its memory limit. Check: kubectl describe pod (reason OOMKilled, exit code 137), whether limits are too low or there's a real memory leak; compare requests/limits vs actual usage in Grafana.

ImagePullBackOff

Wrong image tag/name, private registry auth missing (imagePullSecrets), or registry unreachable from the node.

Pending

Scheduler can't place the pod — usually insufficient CPU/memory on nodes, unsatisfied node affinity/taints, or no PVC available. kubectl describe pod shows the exact scheduling failure reason.

Node NotReady

Kubelet not reporting — check kubelet service status, disk pressure, network plugin (CNI) health, or node resource exhaustion.

kubectl describe pod <name>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
kubectl get pod -o yaml

Monitoring

5-star · Monitoring

175. Explain the Prometheus architecture and metric types.

Source answer:

Application
→
Exporter
→
/metrics
→
Prometheus
→
Alertmanager
→
Grafana
Prometheus
pulls/scrapes
metrics, and collects metrics NOT logs. It's a time-series database.
Counter
Monotonically increasing — e.g. total requests. Use rate()/increase() to make it useful.
Gauge
Goes up/down — e.g. CPU, memory, active connections.
Histogram
Bucketed distribution — e.g. latency; server-side percentile calculation via histogram_quantile().
Summary
Client-calculated latency percentiles — cheaper to query but can't be aggregated across instances.

5-star · Monitoring

176. Write a PromQL query to get the 5-minute request rate and the P99 latency.

Source answer:

```
# request rate over 5m window
rate(http_requests_total[5m])
# error rate %
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100
# P99 latency from a histogram
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
# CPU usage per pod
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)
```

Q: rate() vs irate()?
rate() averages over the whole window — smoother, good for alerting/dashboards. irate() uses only the last two data points — more reactive but spikier, good for fast-moving graphs.
Q: What's a recording rule?
A pre-computed PromQL query saved as a new metric on a schedule — used to speed up expensive/frequent queries (e.g. dashboards, SLO burn rate) instead of recalculating every time.

5-star · Monitoring

177. How does Alertmanager decide who gets paged and when?

Source answer:

Routing
Tree of match rules (by label, e.g. team=platform, severity=critical) sends alerts to the right receiver (Slack, PagerDuty, email).
Grouping
Combines related alerts (same alertname/cluster) into a single notification to avoid alert storms.
Inhibition
Suppresses lower-severity alerts when a related higher-severity one is already firing (e.g. suppress "high latency" if "service down" is firing).
Silencing
Manually mutes alerts for a time window — e.g. during planned maintenance.
Q: How do you avoid alert fatigue?
Alert on symptoms (SLO burn rate, error rate) not causes; use multi-window burn-rate alerts; group related alerts; route by severity; regularly prune noisy/non-actionable alerts. This maps directly to your "40% alert noise reduction" resume point.

5-star · Monitoring

178. How do you build and maintain a good Grafana dashboard?

Source answer:

Data source

Prometheus, Loki, Elasticsearch, CloudWatch, etc. — Grafana visualizes, doesn't store, metrics.

Panel

A single visualization (graph, table, stat, heatmap) backed by a query.

Variable/template

Dropdown (e.g. $namespace, $pod) to make one dashboard reusable across environments/services.

Grafana Alerting

Can alert directly off any data source query, separate from Prometheus/Alertmanager rules.

Good dashboard design mirrors the Four Golden Signals: latency, traffic, errors, saturation — top row of any service dashboard.

Q: Prometheus vs Grafana — who does what?

Prometheus scrapes, stores, and evaluates alert rules on time-series data. Grafana is purely the visualization/dashboarding layer and can also layer its own alerting on top of any data source.

Q: How do you monitor logs alongside metrics?

Loki (or Splunk/ELK) for logs, correlated with Prometheus metrics via shared labels (pod, namespace) — Grafana can show both in one dashboard, and exemplars can link a latency spike directly to a trace.

4-star · Monitoring

179. Your Prometheus is running out of memory / queries are slow — what's wrong?

Source answer:

High cardinality

Too many unique label combinations (e.g. a label with user_id or full URL) — explodes the number of time series and memory usage.

Retention

Local Prometheus storage is typically short-lived (15d default); long-term storage needs remote_write to Thanos/Mimir/Cortex.

Federation

One Prometheus scrapes aggregated metrics from others — used for hierarchical/multi-cluster setups.

Fix for cardinality: drop/relabel high-cardinality labels at scrape time, avoid unbounded label values, use recording rules to pre-aggregate.

5-star · Monitoring

180. Explain latency percentiles and why P99 matters more than average.

Source answer:

Interviewers love asking this — anchor it with a concrete example, not just definitions.

Example: 100 requests

P50

50 requests completed within 100ms.

P95

95 requests completed within 350ms.

P99

99 requests completed within 900ms.

Higher percentiles are more useful operationally — because slow users matter, and averages hide tail latency. A single P99-heavy dependency can dominate real user experience even if the average looks fine.

Kubernetes

5-star · Kubernetes

181. Explain the phases a Pod goes through.

Source answer:
Pending
→
ContainerCreating
→
Running
→
Succeeded / Failed
Pending
Accepted by cluster, not yet scheduled/image not pulled.
Running
Bound to a node, all containers created, at least one running.
Succeeded
All containers terminated successfully (Jobs).
Failed
At least one container terminated with failure.
Unknown
State can't be determined — usually node communication lost.

5-star · Kubernetes

182. What's the difference between an Init Container and a Sidecar?

Source answer:
Init Container
Runs to completion
before
app containers start. Used for setup: wait-for-dependency, config generation, DB migrations.
Sidecar
Runs
alongside
the app container for the pod's whole lifetime. Used for: log shipping, service mesh proxy (Envoy), metrics exporters.

5-star · Kubernetes

183. When would you use a Deployment vs StatefulSet vs DaemonSet?

Source answer:
Deployment
Stateless apps, interchangeable pods, random names, any pod can be replaced by any other.
StatefulSet
Stateful apps needing stable network identity + stable storage per pod (databases, Kafka, Elasticsearch). Ordered, sticky pod names (pod-0, pod-1...).
DaemonSet
Exactly one pod per node — used for node-level agents: log collectors, monitoring agents, CNI plugins.
ReplicaSet
Ensures N replicas of a pod are running; a Deployment manages ReplicaSets for you (you rarely touch ReplicaSets directly).

5-star · Kubernetes

184. When do you use a ConfigMap vs a Secret?

Source answer:
ConfigMap
Non-sensitive config: URLs, feature flags, app settings. Plaintext.
Secret
Sensitive data: passwords, tokens, keys. Base64-encoded (NOT encrypted by default) — pair with encryption at rest / external secret managers (Vault, AWS Secrets Manager, Azure Key Vault) for real security.
Common trap: interviewers ask "is a Secret secure?" — correct answer: base64 is encoding not encryption; real security needs etcd encryption at rest + RBAC + an external secrets manager.

5-star · Kubernetes

185. Explain the three probe types and what happens when each fails.

Source answer:
Liveness
Is the app alive? If it fails → kubelet
restarts
the container.
Readiness
Is the app ready for traffic? If it fails → pod is
removed from Service endpoints
(no restart).
Startup
For slow-starting apps — disables liveness/readiness checks until startup succeeds, preventing premature restart kills during a long boot.
Classic trap question: "Your app is stuck restarting in a loop but isn't actually broken." → usually a liveness probe timeout set too aggressively for a slow-starting app — fix with a startup probe or higher timeouts.

5-star · Kubernetes

186. What's the difference between HPA, VPA, and the Cluster Autoscaler?

Source answer:
HPA
Horizontal Pod Autoscaler — adds/removes
pod replicas
based on CPU/memory/custom metrics.
VPA
Vertical Pod Autoscaler — adjusts a pod's
CPU/memory requests
automatically; usually can't run alongside HPA on the same metric.
Cluster Autoscaler
Adds/removes
nodes
when pods can't be scheduled (Pending) or nodes are underutilized.
They work together: HPA scales pods → if nodes run out of capacity → Cluster Autoscaler adds nodes.

5-star · Kubernetes

187. Explain ClusterIP, NodePort, LoadBalancer, and Ingress.

Source answer:
ClusterIP
Default — internal-only IP, reachable only inside the cluster.
NodePort
Exposes the service on a static port on every node's IP — rarely used directly in production.
LoadBalancer
Provisions a cloud LB (ELB/ALB/Azure LB) pointing at the service — one LB per service, L4 mostly.
Ingress
L7 HTTP(S) routing (host/path-based) in front of multiple services through a single entry point — needs an Ingress Controller (Nginx, ALB Controller) to actually work.

4-star · Kubernetes

188. What's the difference between CNI and CSI?

Source answer:
CNI
Container Network Interface — plugin standard for pod networking (Calico, Cilium, Flannel). Handles pod IPs, network policies.
CSI
Container Storage Interface — plugin standard for storage provisioning (EBS CSI, Azure Disk CSI). Handles PV/PVC dynamic provisioning.

4-star · Kubernetes

189. How do you control which pods land on which nodes?

Source answer:
Taints & Tolerations
Taint = repel pods from a node unless they explicitly tolerate it. One-directional: node says "keep out unless...".
Affinity / Anti-Affinity
Pod expresses a preference: "run me near/away from X". Node affinity = pod-to-node rules. Pod affinity/anti-affinity = pod-to-pod rules (e.g. spread replicas across AZs).
Common real use: taint GPU nodes so only GPU workloads tolerate/schedule there; use pod anti-affinity to spread replicas across zones for HA.

5-star · Kubernetes

190. What happens if you set limits without requests, or vice versa?

Source answer:
Requests
Guaranteed minimum — used by the scheduler to place the pod.
Limits
Hard ceiling — CPU is throttled above it; memory above it → OOMKilled.
Guaranteed
requests == limits for all containers — highest priority, last to be evicted.
Burstable
requests < limits — medium priority.
BestEffort
no requests/limits set — first to be evicted under node pressure.
Trap: no requests set → scheduler doesn't reserve anything → node can be overcommitted → noisy-neighbor problems.

4-star · Kubernetes

191. How does Kubernetes RBAC work, and why use namespaces?

Source answer:
Role / ClusterRole
Defines a set of permissions (verbs on resources) — Role is namespace-scoped, ClusterRole is cluster-wide.
RoleBinding / ClusterRoleBinding
Attaches a Role/ClusterRole to a user, group, or ServiceAccount.
Namespace
Logical isolation boundary — for RBAC scope, ResourceQuota, and organizing teams/environments in one cluster.
ResourceQuota
Caps total CPU/memory/object count a namespace can consume — prevents one team from starving others.

4-star · Kubernetes

192. How do you make sure a rollout or node drain doesn't take your service down?

Source answer:
PodDisruptionBudget
Guarantees a minimum number/percentage of pods stay available during voluntary disruptions (node drains, upgrades).
RollingUpdate
Default strategy — replaces pods gradually, controlled by maxSurge/maxUnavailable. Zero downtime if configured right.
Recreate
Kills all old pods before creating new ones — causes downtime; used when the app can't run two versions simultaneously (e.g. schema-incompatible releases).

4-star · Kubernetes

193. What are the core Kubernetes control plane components and what does each do?

Source answer:
etcd
Distributed key-value store — the single source of truth for all cluster state. Losing etcd = losing the cluster.
kube-apiserver
Front door — all requests (kubectl, controllers) go through it; validates and persists to etcd.
kube-scheduler
Decides which node a new pod runs on, based on resources, affinity, taints.
kube-controller-manager
Runs reconciliation loops (ReplicaSet, Node, Endpoint controllers) to match actual state to desired state.
kubelet
Node agent — talks to the container runtime, reports node/pod status back to the API server.
kube-proxy
Maintains network rules on nodes so Services can route traffic to the right pods.

4-star · Kubernetes

194. What problem does Helm solve, and what are Charts/Values/Releases?

Source answer:
Chart
Packaged, templated set of K8s manifests — the "app definition".
Values.yaml
Configuration input — lets one chart be reused across dev/staging/prod by overriding values.
Release
A deployed instance of a chart with a specific set of values — versioned, so you can rollback.
helm install myapp ./chart -f values-prod.yaml
helm upgrade myapp ./chart --set image.tag=v2
helm rollback myapp 1

5-star · Kubernetes

195. Rapid-fire K8s concepts to have crisp, ready answers for before any interview.

Source answer:
Pod lifecycle
Init Containers vs Sidecar Containers
Deployment vs StatefulSet
DaemonSet
ReplicaSet
ConfigMap vs Secret
Readiness vs Liveness vs Startup Probe
HPA vs VPA vs Cluster Autoscaler
Ingress vs LoadBalancer vs NodePort vs ClusterIP
CNI and CSI
Taints and Tolerations
Affinity and Anti-Affinity
Resource Requests vs Limits, QoS classes
Rolling Update vs Recreate
RBAC, Namespaces, ResourceQuota
PodDisruptionBudget
etcd and control plane components
CrashLoopBackOff / OOMKilled / ImagePullBackOff / Pending — root causes
Helm charts, values, releases
Keep each answer to 2–3 sentences with one concrete example — don't let these ramble.

Linux/Bash

5-star · Linux/Bash

196. What do $0, $1, $#, $@, $? mean in a bash script?

Source answer:
$0
Script name.
$1
First argument.
$2
Second argument.
$#
Argument count.
$@
All arguments.
$?
Exit status of the previous command.
./deploy.sh prod app1
$1 = prod
$2 = app1

5-star · Linux/Bash

197. Production server is misbehaving — what commands do you reach for?

Source answer:

disk
df -h
du -sh * | sort -rh | head
processes
ps -ef
top / htop
kill -9 <pid>
open files / ports
lsof -i :8080
netstat -tulnp
ss -tulnp
tracing a stuck process
strace -p <pid>
scheduling
crontab -l
systemctl status <service>
journalctl -u <service> -f
Q: Disk is full but du shows free space?
Classic trap — a deleted file is still held open by a running process. lsof | grep deleted finds it; restarting/killing that process releases the space.

4-star · Linux/Bash

198. Find all lines with '5xx' errors in a log and count occurrences per status code.

Source answer:

grep "5[0-9][0-9]" access.log | wc -l
awk '{print $9}' access.log | sort | uniq -c | sort -rn
sed -n '100,200p' file.log # print lines 100-200
tail -f app.log | grep --line-buffered ERROR
Q: grep vs awk vs sed — when do you reach for each?
grep = search/filter lines. awk = column-based extraction and simple aggregation. sed = stream editing/substitution. Combine with pipes for log analysis.

3-star · Linux/Bash

199. What does chmod 755 mean?

Source answer:

7 (owner)
rwx — read, write, execute.
5 (group)
r-x — read, execute.
5 (other)
r-x — read, execute.
chmod 644 = owner read/write, group/other read-only — typical for config files. chown user:group file changes ownership.

SRE Fundamentals

5-star · SRE Fundamentals

200. Define SLI, SLO, and SLA — and walk through a real numeric example.

Source answer:

SLI
Service Level
Indicator
— the raw measurement: e.g. % of successful requests, or request latency.
SLO
Service Level
Objective
— the internal target for that SLI, e.g. 99.9% of requests succeed.
SLA
Service Level
Agreement
— the external, often contractual, commitment to the customer, usually looser than the SLO, with a financial penalty if missed.
Worked example — checkout API
SLI:
% of HTTP requests to /checkout returning non-5xx within 500ms.
SLO:
99.9% of requests meet that in a rolling 30-day window.
SLA:
Contract promises customers 99.5% — deliberately looser than the internal SLO, so you breach your own target before you ever breach the customer contract.
Math:
30 days = 43,200 minutes. 0.1% allowed failure = 43.2 minutes of "bad" time per month (your error budget window) before the SLO itself is breached.
Interviewers like hearing that SLO should always be stricter than SLA — the SLO is your early warning system before you ever breach a paid contract.

5-star · SRE Fundamentals

201. What is an error budget, and how do multi-window burn-rate alerts work?

Source answer:

Worked example
SLO = 99.9% → allowed downtime = 0.1% → over 30 days that's ~43 minutes. That 43 minutes is the
error budget
.
If the error budget is exhausted → freeze feature releases, redirect the team to reliability work until budget recovers.
Q: What's "burn rate"?
How fast you're consuming the error budget relative to a sustainable pace. A burn rate of 10x means you'd exhaust a 30-day budget in 3 days if it continued — that's what pages you, not the raw SLO breach itself.
Q: Why use multi-window (e.g. 5m + 1h) burn-rate alerts instead of a single threshold?
A short window catches fast, severe outages quickly; a longer window filters out noise from brief blips. Combining both reduces false positives while still catching real incidents fast — directly addresses alert fatigue.

5-star · SRE Fundamentals

202. What are the Google SRE Four Golden Signals?

Source answer:

Latency
Time to service a request — separate successful vs failed request latency.
Traffic
Demand on the system — requests/sec, throughput.
Errors
Rate of failed requests — explicit (5xx) and implicit (wrong content) failures.
Saturation
How "full" the system is — CPU, memory, queue depth, connection pool usage.
Mnemonic:
LTES
. Use these as the default lens for any "what would you monitor for X" question.

4-star · SRE Fundamentals

203. What is 'toil' in the SRE sense, and how do you reduce it?

Source answer:

Toil

Manual, repetitive, automatable work that scales linearly with service growth and has no lasting engineering value — e.g. manually restarting a service every time it hits a known issue.

Google SRE guidance: cap toil at ~50% of an SRE's time; the rest goes to engineering work that reduces future toil — automation, tooling, self-healing systems. Good answer to "how do you decide what to automate?"

Java/App Support

4-star · Java/App Support

204. Explain WAR vs JAR, Tomcat vs WildFly, heap dump vs thread dump, GC, JVM, classloader.

Source answer:

WAR vs JAR

WAR = deployable web app archive (needs a servlet container); JAR = general packaged Java library/app.

Tomcat vs WildFly

Tomcat = lightweight servlet container; WildFly = full Java EE application server.

Heap dump

Snapshot of memory/objects — used to debug memory leaks.

Thread dump

Snapshot of all thread states — used to debug hangs/deadlocks/high CPU.

GC

Garbage Collection — reclaims unused heap memory; a bad GC pattern shows up as periodic latency spikes ("stop-the-world" pauses).

JVM

Java Virtual Machine — runs compiled bytecode.

Classloader

Loads Java classes into the JVM at runtime.

Frequently asked of SREs supporting Java workloads — expect at least one of these per interview.

Docker

5-star · Docker

205. How would you optimize a slow, bloated Dockerfile?

Source answer:

Layers

Each instruction (RUN/COPY/ADD) creates a cached layer. Order matters — put least-frequently-changing steps (e.g. dependency install) before frequently-changing ones (e.g. COPY source code) to maximize cache hits.

Multi-stage build

Use one stage to compile/build, then COPY only the final artifact into a slim runtime image — drastically reduces final image size and attack surface.

```
# multi-stage example
FROM maven:3.9 AS build
COPY . .
RUN mvn clean package
FROM eclipse-temurin:17-jre-alpine
COPY --from=build /app/target/app.jar .
```

Q: ENTRYPOINT vs CMD?

CMD provides default args, easily overridden at docker run. ENTRYPOINT fixes the main command; args passed at runtime append to it. Often combined: ENTRYPOINT ["java","-jar","app.jar"] with CMD for default flags.

4-star · Docker

206. What's actually running my containers under the hood?

Source answer:

Docker

The end-to-end developer tool/daemon (build, push, run) — historically also the runtime.

containerd

Lower-level container runtime (a CNCF project) that manages the container lifecycle — this is what Kubernetes actually talks to via CRI now, not the Docker daemon.

runc

The lowest-level OCI-compliant tool that actually creates and runs the container using Linux namespaces/cgroups.

Interview-relevant: Kubernetes deprecated dockershim — modern clusters use containerd (or CRI-O) directly, not the Docker daemon.

Terraform/IaC

5-star · Terraform/IaC

207. What is Terraform state, why does it need locking, and what is drift?

Source answer:
State file
terraform.tfstate — Terraform's record of what it thinks exists in the real world; maps resources to config.
Remote backend
State stored remotely (S3 + DynamoDB lock table, Azure Storage, Terraform Cloud) — enables team collaboration and locking.
State locking
Prevents two people from running apply simultaneously and corrupting state.
Drift
Real infrastructure changed outside Terraform (manual console edit) — state no longer matches reality; terraform plan reveals it.
terraform plan
terraform apply
terraform state list
terraform import <resource> <id>
terraform refresh

4-star · Terraform/IaC

208. How do you manage the same Terraform config across dev/staging/prod?

Source answer:
Module
Reusable, parameterized bundle of resources — write once, call with different variables per environment.
Workspace
Lightweight way to reuse the same config with separate state files (e.g. dev/prod) — best for small variations, not a substitute for proper module/directory separation on complex environments.
Common answer pattern: separate directories/repos per environment calling shared modules, rather than relying solely on workspaces, for stronger isolation and blast-radius control.

Git/GitHub Actions

4-star · Git/GitHub Actions

209. When would you rebase vs merge, and what does cherry-pick do?

Source answer:
Merge
Combines branches, preserves full history, creates a merge commit. Safe for shared/public branches.
Rebase
Replays your commits on top of another branch — linear, cleaner history, but rewrites commit hashes. Never rebase a branch others are already using.
Cherry-pick
Applies one specific commit from another branch onto your current branch — useful for hotfix backports.

4-star · Git/GitHub Actions

210. How do GitHub Actions and Jenkins compare?

Source answer:
GitHub Actions
YAML-defined, event-triggered (push/PR/schedule), hosted or self-hosted runners, tightly integrated with GitHub. Faster to set up, less infra to manage.
Jenkins
Self-hosted, highly pluggable (1000s of plugins), Groovy pipelines, more control/customization but more operational overhead (upgrades, plugin management, scaling agents).

Cloud (AWS/Azure)

4-star · Cloud (AWS/Azure)

211. Explain the difference between an IAM user, role, and policy.

Source answer:
Policy
JSON document defining permissions — what actions are allowed/denied on which resources.
User
A permanent identity (person or service) with long-lived credentials.
Role
Temporary identity assumed by a user/service — no long-lived credentials; preferred for workloads (e.g. EC2/EKS pods assuming a role via IRSA) over embedding access keys.
Strong answer: "prefer roles over static access keys" — signals security maturity.

3-star · Cloud (AWS/Azure)

212. What's a VPC/VNet, and what are subnets, route tables, and security groups?

Source answer:
VPC / VNet
Your isolated private network in the cloud.
Subnet
A segment of the VPC's IP range, tied to an AZ; public (has route to Internet Gateway) or private.
Route table
Controls where subnet traffic goes.
Security Group / NSG
Stateful, instance-level firewall — allow rules only (in AWS SGs); NACLs/Azure NSGs can also deny, and are subnet-level and stateless.

Other / Misc

4-star · Other / Misc

213. What is GitOps, and how does ArgoCD implement it?

Source answer:
GitOps
Git as the single source of truth for desired infra/app state; an operator continuously reconciles the cluster to match Git — no manual kubectl apply in prod.
ArgoCD
Watches a Git repo, auto-syncs (or requires manual sync) the cluster to match, shows drift/diff in its UI, supports automated rollback by reverting Git.
Good soundbite: "the cluster state should always be reproducible by replaying Git history" — shows you understand the philosophy, not just the tool.

3-star · Other / Misc

214. What's the point of chaos engineering, and how have you used it?

Source answer:
Deliberately inject failure (kill a pod, add latency, saturate CPU) in a controlled way to validate that your monitoring, alerting, and failover actually work — before a real outage proves they don't.
Q: How do you run a chaos experiment safely?
Start in staging or with a small blast radius in prod, define a clear hypothesis and success/abort criteria upfront, monitor Golden Signals live, and have a one-click rollback/abort ready.

3-star · Other / Misc

215. What ITIL concepts come up in a ticketing/ServiceNow-driven environment?

Source answer:
Incident Management
Restore service ASAP — matches the Incident card above.
Change Management
Controlled process for making changes — Standard (pre-approved, low-risk), Normal (needs CAB approval), Emergency (fast-tracked for active incidents).
Problem Management
Root cause analysis to prevent recurrence — matches the Problem card above.
CAB
Change Advisory Board — reviews and approves higher-risk changes.

3-star · Other / Misc

216. How should you structure answers to 'tell me about a time...' questions?

Source answer:
Situation
Brief context — system, scale, stakes.
Task
What you were specifically responsible for.
Action
What YOU did, step by step — this is the bulk of the answer.
Result
Quantified outcome — tie back to your resume metrics (uptime, MTTR, alert noise reduction).
Have 4-5 STAR stories ready covering: a major outage you resolved, a conflict with a teammate/manager, a time you automated away toil, a time you pushed back on a risky change, and a mistake you made and how you recovered.

Production Scenarios

5-star · Production Scenarios

217. You're paged at 2am — users can't log in to production. Walk me through what you do, in order.

Source answer:

Ack the page
→
Check status dashboard
→
Scope: all users or subset?
→
Recent deploy/change?
→
Check auth service health
→
Check dependency (DB, IdP, cache)
→
Mitigate (rollback/restart/failover)
→
Verify recovery
→
Communicate status
→
Postmortem next day

Say this out loud: "First I stabilize, then I root-cause." Interviewers are checking you don't dive into deep debugging before mitigating customer impact. Also mention: update the status page and notify stakeholders early, even before full RCA.

Q: What if the fix isn't obvious in the first 10 minutes?

Roll back to the last known-good deploy immediately if a recent change correlates with the start time — don't wait for full root cause. Rollback is a mitigation, not the investigation.

5-star · Production Scenarios

218. You just deployed and error rates jumped. Do you roll back or push a fix forward?

Source answer:

Roll back

Default choice when impact is severe/customer-facing and the previous version was stable — fastest path to recovery, buys time to investigate calmly.

Roll forward

Only when the fix is trivial, well-understood, and rollback itself is risky (e.g. destructive DB migration already ran) — otherwise it's slower and riskier under pressure.

Strong answer: "I default to rollback for speed and safety, and only roll forward if rollback isn't actually possible — e.g. an irreversible migration already applied." Mention feature flags as a way to avoid this dilemma entirely next time.

5-star · Production Scenarios

219. Your app is throwing 'connection pool exhausted' errors in production. How do you troubleshoot and fix it?

Source answer:

Confirm via app/DB metrics
→
Check active vs idle connections
→
Long-running/slow queries?
→
Recent traffic increase?
→
Connection leak in code?
→
Short-term: bump pool size / kill long queries
→
Long-term: fix leak, add pooler (PgBouncer), query optimization

Good follow-up detail: distinguish "pool too small for legitimate load" (scaling problem) from "connections not being released" (leak/bug) — the fix is completely different.

5-star · Production Scenarios

220. Traffic just 10x'd unexpectedly and the service is degrading. What do you do in real time, and how would you have prepared?

Source answer:
Check Golden Signals
→
Is autoscaling keeping up?
→
Identify the bottleneck (app/DB/cache/LB)
→
Manually scale if HPA is too slow
→
Shed non-critical load / enable rate limiting
→
Cache aggressively if reads are hot
Preparation angle interviewers want to hear: load testing ahead of known events, pre-warmed autoscaling (min replicas raised in advance), and a documented "break glass" scale-up runbook.

5-star · Production Scenarios

221. A node is at 100% disk and pods are getting evicted. Walk me through it.

Source answer:
df -h
du -sh /var/log/* | sort -rh | head
kubectl describe node <node> | grep -A5 Conditions
Common causes
Unrotated logs, orphaned container images/layers, core dumps, a runaway app writing to disk, unbounded log volume from a crash loop.
Immediate
Free space (prune images, rotate/ship logs, delete cores) or cordon+drain the node so new pods land elsewhere.
Prevention
Log rotation policies, shipping logs off-node quickly (Fluentd/Filebeat), disk pressure alerts before it's critical, ephemeral-storage limits/requests on pods.

4-star · Production Scenarios

222. A certificate expired and took down a production endpoint. How do you fix it now, and prevent it happening again?

Source answer:
Immediate
Issue/renew the cert (cert-manager, Let's Encrypt, internal CA), redeploy/reload the affected Ingress or service, verify with curl -v / openssl s_client.
Prevention
Automated renewal (cert-manager with auto-renew well before expiry), expiry-monitoring alerts (e.g. 30/14/7 days out), and an inventory of all certs so none are "silent."
This is a classic "should never have been a surprise" incident — interviewers want to hear the systemic fix (monitoring/automation), not just the manual renewal.

5-star · Production Scenarios

223. An entire AWS region/AZ goes down. What's your failover process?

Source answer:
Confirm scope via provider status page
→
Trigger DNS/traffic failover (Route53 health checks)
→
Verify secondary region capacity
→
Check data replication lag (DB, cache)
→
Redirect traffic
→
Monitor for split-brain / stale reads
→
Fail back once region recovers
If asked "have you actually done this" and you haven't at that scale — be honest, then bridge: describe an AZ-level failover or DR drill you have done, and the same principles apply at region scale.

5-star · Production Scenarios

224. You upgraded the cluster version and now some workloads are misbehaving. How do you handle it, and how would you have done the upgrade more safely?

Source answer:
Diagnose
Check for deprecated/removed API versions (common breakage source), CNI/CSI driver compatibility, changed default behaviors in the new version's changelog.
Mitigate
Roll back the control plane if the managed service allows it, or patch the affected manifests to the new API version and redeploy.
Safer process for next time: read the version's deprecation notices first, upgrade a non-prod cluster and run the full workload suite against it, upgrade one node pool/AZ at a time, keep a rollback plan for the control plane version.

5-star · Production Scenarios

225. An API key or DB password just got committed to a public repo. What do you do?

Source answer:
Rotate/revoke the credential immediately
→
Assume it's compromised — check for misuse in logs
→
Purge from git history (not just a new commit)
→
Redeploy with the new credential
→
Add prevention: pre-commit secret scanning, gitleaks/truffleHog in CI
Key point interviewers check for: rotate first, before anything else — a new commit removing the secret does NOT make it safe, since it's still in git history and possibly already scraped.

4-star · Production Scenarios

226. A few slow queries are causing timeouts that cascade into a full outage. How do you stop the bleeding and fix the root cause?

Source answer:
Stop the bleeding
Identify and kill the offending queries, add/adjust a statement_timeout, scale read replicas if it's read-heavy, add a circuit breaker so one slow dependency doesn't take down the whole app.
Root cause
Missing index, a bad query plan after a data volume change, a new feature querying without pagination, or lock contention — check slow query logs / EXPLAIN ANALYZE.
Mention "cascading failure" explicitly and how a circuit breaker / bulkhead pattern (isolating thread pools per dependency) prevents one slow dependency from exhausting all app threads.

4-star · Production Scenarios

227. You've confirmed a slow memory leak in a production service. Restarting fixes it temporarily — what's your full approach?

Source answer:
Short-term: scheduled/automated restarts as a stopgap
→
Capture heap dump before a restart
→
Analyze for growing object counts
→
Bisect recent releases to find the introducing change
→
Fix and validate under load before rollout
Explicitly say restart is a mitigation, not a fix — interviewers want to know you won't let "just restart it" become the permanent answer.

5-star · Production Scenarios

228. A pipeline just auto-deployed a broken config to prod. How do you respond, and how do you prevent this class of incident?

Source answer:
Respond
Roll back via the pipeline/GitOps revert (revert the Git commit, let ArgoCD/pipeline redeploy the last good state) rather than manual kubectl edits that drift from source of truth.
Prevent
Config validation/linting in CI before merge, canary/staged rollout so bad config only hits a small % of traffic first, required review on config changes touching prod, automated smoke tests post-deploy that auto-rollback on failure.

4-star · Production Scenarios

229. A third-party API you depend on is slow/erroring, and it's dragging your whole service down. What do you do?

Source answer:
Immediate
Circuit breaker trips to fail fast instead of hanging; serve degraded/cached responses if possible (graceful degradation) rather than a full outage.
Structural fix
Timeouts + retries with backoff (bounded, not infinite retries — avoid retry storms), bulkhead isolation so this dependency's failures can't exhaust shared thread/connection pools.
Good line: "a hard dependency on someone else's uptime should degrade gracefully, not become my outage."

4-star · Production Scenarios

230. Someone ran a bad DELETE/migration against the production database. What now?

Source answer:
Stop further writes if possible
→
Assess blast radius (which tables/rows/time range)
→
Restore from latest backup / PITR to a separate instance
→
Reconcile/replay any writes since the backup
→
Cut back over
→
Postmortem: why did this run unguarded against prod?
Prevention angle they're listening for: point-in-time recovery enabled, migrations require a dry-run/review step, destructive migrations gated behind extra approval, least-privilege DB access so most engineers can't run raw DELETE against prod at all.

4-star · Production Scenarios

231. HPA should have scaled the service up but didn't, and it fell over under load. How do you debug this after the fact?

Source answer:
kubectl describe hpa <name>
kubectl get hpa -w
kubectl top pod
Common causes
Metrics server / custom metrics pipeline was itself down or lagging; requests not set on containers (HPA has nothing to calculate % against); maxReplicas set too low; Cluster Autoscaler couldn't add nodes fast enough even though HPA tried to scale pods.
Good answer differentiates "HPA didn't decide to scale" (metrics/config problem) from "HPA scaled pods but there was no node capacity" (cluster autoscaler problem) — check both.

3-star · Production Scenarios

232. You see a massive, unusual spike in traffic from a narrow set of IPs and error rates climbing. Attack or legitimate spike — how do you tell, and what do you do?

Source answer:
Distinguish
Check request patterns (single endpoint hammered vs. broad usage), geographic/IP concentration, user-agent anomalies, and whether it correlates with a marketing event or news mention.
Mitigate
Rate limiting / WAF rules, enable a CDN/DDoS protection layer (CloudFlare, AWS Shield), block/challenge suspicious IP ranges, scale up temporarily to absorb legitimate portion of traffic.
Say you'd loop in security/on-call leadership for anything that looks like a real attack — this isn't a solo call in most orgs.

Jenkins

5-star · Jenkins

233. Walk me through the structure of a Jenkins declarative pipeline — the blocks you'd actually use.

Source answer:

pipeline { }
Top-level wrapper — everything lives inside this.
agent
Where it runs: any, none, label('docker'), or a docker/kubernetes agent block.
environment
Env vars available to all stages — e.g. credentials(), image tags.
parameters
Build-time inputs — string/choice/boolean params a user fills in when triggering manually.
options
Pipeline behavior — timeout, retry, disableConcurrentBuilds, buildDiscarder.
triggers
cron(), pollSCM(), or upstream job triggers.
stages / stage
The actual pipeline steps, shown in the Jenkins UI as visual blocks.
steps
Shell/sh commands or plugin steps inside a stage.
when
Conditional stage execution — e.g. only run "Deploy" when branch == 'main'.
post
always / success / failure / cleanup — runs after stages regardless of outcome (notifications, cleanup).

```
pipeline {
agent any
environment { IMAGE_TAG = "v${'$'}{BUILD_NUMBER}" }
stages {
stage('Checkout') { steps { checkout scm } }
stage('Build') { steps { sh 'mvn clean package' } }
stage('Test') { steps { sh 'mvn test' } }
stage('SonarQube'){ steps { sh 'mvn sonar:sonar' } }
stage('Docker Build & Push') {
steps { sh 'docker build -t myapp:${'$'}{IMAGE_TAG} . && docker push myapp:${'$'}{IMAGE_TAG}' }
}
stage('Trivy Scan') { steps { sh 'trivy image myapp:${'$'}{IMAGE_TAG}' } }
stage('Deploy') {
when { branch 'main' }
steps { sh 'kubectl set image deploy/myapp app=myapp:${'$'}{IMAGE_TAG}' }
}
}
post { always { cleanWs() } failure { slackSend "Build failed" } }
}
```

Memorize this shape — nearly every "walk me through your Jenkinsfile" question maps directly onto it: Checkout → Build → Test → Static analysis → Image build/scan → Deploy → post notification.

4-star · Jenkins

234. How do you scale Jenkins across teams and keep pipelines DRY and secure?

Source answer:

Master/Controller vs Agent
Controller schedules and manages jobs; agents (static or dynamic — e.g. Kubernetes pod agents) actually execute the steps, keeping the controller light.
Shared Library
Reusable Groovy pipeline code hosted in its own repo, imported with @Library — avoids copy-pasting the same stages across every team's Jenkinsfile.
Credentials Store
Jenkins-managed secrets injected via credentials() / withCredentials{} — never hardcoded, scoped per-folder or global.
Dynamic agents
Kubernetes plugin spins up a pod-per-build agent on demand, then tears it down — avoids idle static VM cost and gives clean, reproducible build environments.
Q: How do you keep a long-running Jenkins pipeline from blocking others?
disableConcurrentBuilds() where needed, parallel {} blocks for independent stages, and dynamic (ephemeral) agents so builds don't compete for a fixed pool.

GitHub Actions

5-star · GitHub Actions

235. Walk me through the structure of a GitHub Actions workflow YAML.

Source answer:

on
Trigger — push, pull_request, schedule (cron), workflow_dispatch (manual), or workflow_call (reusable workflow).
jobs
One or more named jobs, each running on a fresh runner by default and in parallel unless linked with needs.
runs-on
The runner — ubuntu-latest, self-hosted, etc.
steps
Sequential actions inside a job — either uses: (a marketplace/custom action) or run: (a shell command).
needs
Declares job dependency order — e.g. deploy needs: [build, test].
strategy.matrix
Runs the same job across multiple combinations (Node versions, OSes) in parallel.
secrets
Repo/org-level encrypted values, referenced as ${'$'}{{ secrets.MY_SECRET }}, never printed to logs.
name: CI/CD
on:
push: { branches: [main] }
pull_request: { branches: [main] }
jobs:
build:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Build & test
run: mvn clean package
- name: Build & push image
run: |
docker build -t ghcr.io/org/app:${'$'}{{ github.sha }} .
docker push ghcr.io/org/app:${'$'}{{ github.sha }}
deploy:
needs: build
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
steps:
- name: Update image tag in GitOps repo
run: ./scripts/bump-manifest.sh ${'$'}{{ github.sha }}
Notice the deploy job doesn't kubectl apply directly — in a GitOps setup it just updates the manifest repo and lets ArgoCD sync. This distinction is exactly what interviewers probe for.

4-star · GitHub Actions

236. How do you avoid duplicating the same steps across 10 repos, and speed up slow builds?

Source answer:

Reusable workflow
A workflow with on: workflow_call, invoked from other repos with uses: org/repo/.github/workflows/ci.yml@main — the DRY mechanism, analogous to Jenkins Shared Libraries.
Composite action
Bundles a sequence of steps into one reusable custom action — smaller-grained than a full reusable workflow.
actions/cache
Caches dependencies (node_modules, .m2, pip) between runs keyed by a hash of the lockfile — cuts build time significantly.
actions/upload-artifact / download-artifact
Passes build output (jar, coverage report) between jobs in the same workflow run.
Environments
GitHub Environments (staging/production) can require manual approval before a job runs — the built-in equivalent of a deployment gate.

GitOps & ArgoCD

5-star · GitOps & ArgoCD

237. What actually makes something 'GitOps' rather than just 'CI/CD with a git repo'?

Source answer:

Push model (traditional CI/CD)

The pipeline (Jenkins/GitHub Actions) has direct cluster credentials and runs kubectl apply / helm upgrade itself at the end of the pipeline.

Pull model (GitOps)

An in-cluster operator (ArgoCD) continuously watches a Git repo and pulls/applies changes itself — the CI pipeline never touches the cluster directly, it only updates Git.

Dev merges code

→

CI builds & pushes image

→

CI updates image tag in manifest repo

→

ArgoCD detects diff

→

ArgoCD syncs cluster to match Git

Key interview line: "no one has kubectl write access to prod — the only way to change the cluster is a Git commit." That's the actual definition, not just "we use Git."

Q: Why is pull safer than push?

No long-lived cluster credentials sitting in CI; the cluster is the source of truth-checker against Git rather than a one-way blind push; drift is automatically detected and can be auto-corrected.

5-star · GitOps & ArgoCD

238. How is an app configured in ArgoCD, and how do you manage many apps at scale?

Source answer:

Application (CRD)

ArgoCD's core object — points at a Git repo/path (source) and a target cluster/namespace (destination); this is what you actually define per app.

Sync policy: Manual

ArgoCD shows drift/diff but waits for a human to click Sync — common for production.

Sync policy: Automated

Auto-applies Git changes as soon as detected; can combine with prune (delete removed resources) and selfHeal (revert manual cluster drift back to Git automatically).

App of Apps

One root ArgoCD Application whose "source" is itself a folder of other Application manifests — lets you manage dozens of apps/environments declaratively from one entry point.

Sync waves

Annotation-based ordering (argocd.argoproj.io/sync-wave) so dependencies (e.g. a CRD or namespace) apply before the resources that need them.

Health checks

ArgoCD reports Healthy/Progressing/Degraded per resource — e.g. a Deployment is "Progressing" until all replicas are ready.

apiVersion: argoproj.io/v1alpha1

kind: Application

spec:

source: { repoURL: ..., path: manifests/prod, targetRevision: main }

destination: { server: https://kubernetes.default.svc, namespace: myapp }

syncPolicy: { automated: { prune: true, selfHeal: true } }

Q: How do you roll back a bad release with ArgoCD?

git revert the commit that changed the manifest/image tag — ArgoCD detects the diff and syncs the cluster back automatically. You can also use ArgoCD's own rollback to a previous synced revision, but reverting Git is the "correct" GitOps way since it keeps Git as the single source of truth.

Q: selfHeal — what problem does it solve?

If someone manually kubectl edits a live resource (config drift), selfHeal detects the mismatch against Git and automatically reverts it — enforcing that Git truly is the only way to make lasting changes.

Questions to Ask the Interviewer

1. What does the on-call rotation actually look like day to day — how many people, how often does someone get paged overnight?

2. What's the current state of SLIs/SLOs at SearchStax — defined and tracked, or still informal?

3. Since the role reports into Infrastructure under Wolfgang — how does the SRE function collaborate with the core Solr/search engineering team on reliability-affecting changes?

4. What's the biggest reliability challenge the team is dealing with right now?

5. How is success measured for this role in the first 6 months?

6. Given the multi-cloud footprint (AWS/Azure/GCP) — is a given customer's deployment usually on one cloud, or does the SRE team operate across all three regularly?

7. What does the incident postmortem process look like today?

Recommended use: For the live interview, prioritize the original DevOn 102-question round-2 set first, then use the SearchStax technical/behavioral scenarios and the broader SRE recall handbook for deeper follow-ups.